

STAGED IMPLEMENTATION SPECIFICATION

Cakerdesk V1

可靠的 Agent Runtime

从零开始的分阶段实现规格（研究报告）

版本：V1.0

日期：2026-09-29

三条主线：崩溃恢复 · 工具执行语义 · Skill/工具权限硬约束

组成：Go (api / reaper / sandboxd) · Python LangGraph 内核 · PostgreSQL · Redis
(仅 Pub/Sub) · Docker

节奏：6 个阶段，19 个小步；每一步都能跑、能测、能演示，然后提交

本报告取代 docs/report/cakerdesk.html（大而全版本）和旧仓库的 P0。旧 P0 不迁移代码，只借鉴其中几样工程实践（见 1.3）。凡本报告给出 SQL、接口字段、状态转换、阈值或验收用例的地方，实现必须一致；要偏离，先改本报告。

目录

1 这份报告怎么用

2 定位与范围

3 总体架构

4 技术选型

5 全局约定

6 阶段总览

7 阶段 A：地基（A1–A3）

8 阶段 B：最小运行闭环（B1–B4）

9 阶段 C：运行可靠性（C1–C4）

10 阶段 D：工具网关与沙箱（D1–D4）

11 阶段 E：Skill 权限与最小上下文（E1–E3）

12 阶段 F：破坏测试、演示与文档（F1）

13 数据模型汇总（最终 DDL）

14 接口汇总

15 故障模型与恢复矩阵

16 破坏测试清单

17 演示脚本

18 面试要点

19 决策记录

附A 错误码

附B 配置项与默认值

附C 术语表

附D 按步骤的学习资料

1 这份报告怎么用

1.1 为什么从头来

旧 P0 一次性做了多租户控制平面：租户、成员、控制台会话、四种角色、RBAC、7 张表的 FORCE RLS、密钥加密、审计、配额，外加一个带着 17 个工具和记忆/Goal 字段的 AgentVersion schema。它的问题不是写得差，而是**方向不对、一次太多**：

- V1 的核心是 runtime（租约、fencing、checkpoint、工具语义、沙箱），P0 里几乎没有一行服务于这条线。
- 多租户层会给之后的每一张表、每一条查询加税（tenant_id、GRANT、RLS policy、`WithTenant`），调试 fencing 时还会混进权限类故障。
- 面试官打开仓库先看到的是 SaaS 后台，而不是 runtime。

所以 V1 从空仓库开始，每一步只引入一个新概念，并且在进入下一步前把它测到能被破坏测试证明。

1.2 每一步的固定结构

第 7-12 章的每个小步（例如 **C3 Checkpoint fencing**）都按同一个模板写：

栏目	含义
目标	一句话：这一步结束时系统多了什么能力。
学习	这一步要搞懂的概念。先读懂再写，附 D 给出对应资料。
产出	新增/修改的文件、迁移、接口、代码要点（SQL 与关键函数签名写到可以照抄的粒度）。
测试	必须新增的自动化测试，名字固定，方便破坏测试引用。
完成标准	绿框里的每一条都满足才算完成，然后打 tag <code>step-XX</code> 提交。
本步不做	灰框。明确推迟到哪一步，防止提前堆功能。
坑	这一步最容易踩的问题。

1.3 从旧 P0 借鉴什么（只借鉴做法，不拷代码）

做法	在 V1 里的样子	引入步骤
goose 管迁移、SQL 优先	迁移文件嵌入二进制， <code>cakdesk migrate up</code> 。	A2
archtest	用 <code>go list</code> 检查：只有 <code>internal/sandboxd</code> 能 import Docker SDK； <code>internal/kernelapi</code> 以外不能 import 内部 API 处理器。	A1 / D2
lint-stubs	<code>grep</code> 禁止 <code>TODO</code> 、 <code>FIXME</code> 、 <code>NotImplemented</code> 、 <code>panic("unimplemented")</code> 、 <code>pass # stub</code> 进入主分支。	A1
sabotage runner	每条破坏补丁都必须让指定测试失败。	C1 起逐步加，F1 收口
Mock LLM 场景	重写，修掉旧版三个 bug： <code>tool_call_id</code> 重复、 <code>arguments</code> 不是 JSON、错误注入没有次数。	A3
Agent/AgentVersion 不可变	保留概念，schema 缩到 6 个工具和一组限额。	A2

1.4 使用规则

- 1 **严格按顺序**。后一步依赖前一步的测试基线。跳步会被迫写桩。

- 2 **一步一个 tag。**每步完成打 `step-A1 ... step-F1`，README 的“进度”一栏同步勾选。面试时可以按 tag 讲演进。
- 3 **先测后讲。**每个“可靠性声明”都要有一条测试和一条破坏补丁支撑，否则不写进 README。
- 4 **不加报告外的功能。**想加的东西写进 `docs/later.md`，V1 完成后再议。

2 定位与范围

2.1 一句话定位

Cakerdesk 是一个**面向长任务的可靠 Agent Runtime**：把一个 LLM Agent 的执行放进数据库驱动的租约体系里，使它在 worker 被 `kill -9`、网络抖动、工具执行到一半中断、模型乱调工具时，仍然**能续跑、不重复产生副作用、不越权**，并且整个过程可以从事件流和数据库里完整复盘。

2.2 三条主线

主线	要证明的事	对应步骤
崩溃恢复	任何进程随时被杀，Run 最终都会被别的 worker 接手并从最近的 checkpoint 续跑；被接管的旧 worker（僵尸）任何写入都会被拒绝，包括它对 checkpoint 的写入。	B2、C1-C4
工具执行语义	每次工具调用在执行前先持久化 <code>started</code> ；恢复时已完成的调用直接重放结果；幂等工具重跑；非幂等工具报告 <code>interrupted</code> 交给模型决定。不宣称 <code>exactly-once</code> 。	D1、D2、D4
权限硬约束	模型能调用的工具 = Agent 配置的工具 $\cap$ (基线工具 $\cup$ 已加载 Skill 声明的 <code>allowed-tools</code>)，由 Go 网关在执行点检查；bash 在无网络、只读根文件系统、降权的容器里运行。	D3、E1

2.3 V1 做什么

- Agent 与不可变 AgentVersion；Session（拥有工作区与容器）；Run（绑定版本、快照配置）。
- PostgreSQL 作为队列：`FOR UPDATE SKIP LOCKED` 领取、租约、心跳、attempt 作为 fencing token、reaper 回收。
- LangGraph 内核，`thread_id = run_id`，checkpoint 写入带数据库级 fencing。
- 取消与超时是状态转换，不是进程信号。
- 事件：持久化、按 Run 单调 seq、Redis Pub/Sub 实时推送、SSE 支持 `Last-Event-ID` 断点续传。
- 工具网关：6 个工具，`(run_id, tool_call_id)` 幂等，执行语义如 2.2。
- 每个 Session 一个 Docker 容器，bash 用进程组超时杀。
- Skill：`SKILL.md` + `allowed-tools`，`load_skill` 激活，只读挂载。
- 最小上下文：LLM 重试、预算、循环保护、压缩。
- Mock LLM，使全部测试和演示不需要真实 API Key；也可以切到任意 OpenAI 兼容端点。

2.4 V1 明确不做

不做	理由 / 面试时怎么说
多租户、RBAC、RLS	单用户 runtime。扩展路径：每表加 <code>tenant_id</code> + RLS，内部 API 用 <code>BYPASSRLS</code> 角色。
跨会话记忆、向量检索	与可靠性主线无关；长任务靠上下文压缩和工作区文件承载状态。
Goal 续跑链、定时任务、审批、子 Agent	都是“等待-恢复”类功能，V1 的 Run 状态机已为其留出形状（新增状态即可），但不实现。
Redis Streams / outbox / Kafka	PostgreSQL 领取足够且只有一个事实来源；Redis 只做实时推送，丢了不影响正确性。
控制台前端	用 curl、SSE 命令行客户端和 SQL 演示。有余力再做一个只读时间线页面。
Token 级流式输出	事件粒度是一条消息/一次工具调用；流式与可靠性无关。
密钥管理、审计表	LLM Key 走环境变量；事件表本身就是审计轨迹。

不做	理由 / 面试时怎么说
Kubernetes、多副本 api	单机 docker compose + 本机进程。设计上 api 是无状态的（除进程内在飞调用表，见 D4）。

2.5 信任边界

V1 是单用户系统：**Skill 是可信代码**（由用户自己放进 `skills/` 目录），**模型的行为不可信**。沙箱防御的是模型发出的命令（删文件、联网外传、fork 炸弹、越出工作区），不防御恶意 Skill 作者。这一点写进 README，面试时主动说明。

3 总体架构

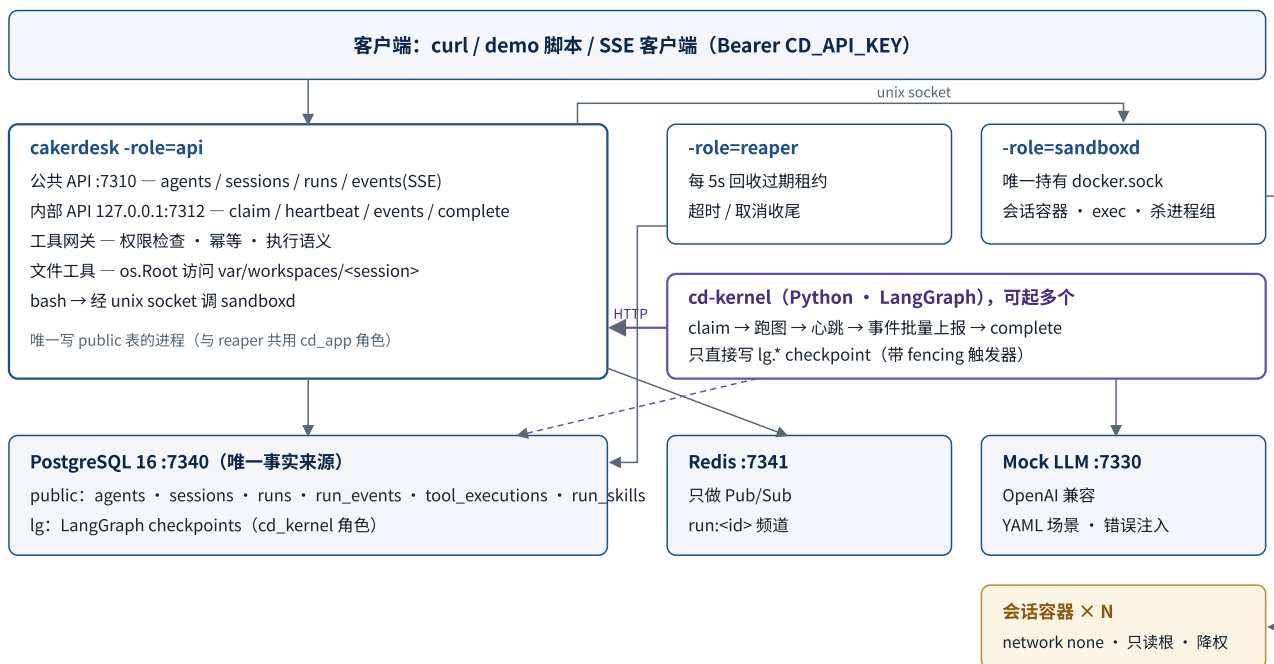


图 3-1 进程与存储。实线：同步调用；虚线：内核直接写 checkpoint（受数据库触发器 fencing）。

3.1 进程与职责

进程	端口/通道	职责	DB 角色
cakdesk - role=api	:7310 公共; 127.0.0.1:7312 内部	公共 REST/SSE; 内部 API (领取、心跳、事件、完成); 工具网关与文件工具; 调用 sandboxd 执行 bash。	cd_app
cakdesk - role=reaper	无	周期回收过期租约 (重新排队或失败)、处理超时与 “已请求取消但 worker 已死” 的 Run, 并写系统事件。	cd_app
cakdesk - role=sandboxd	unix socket var/run/sandboxd.sock (0600)	唯一接触 Docker 的进程: 创建/复用/清理会话容器, exec 命令, 超时杀进程组。没有数据库连接。	无
cd-kernel	出站 HTTP → 7312; 出站 → LLM	领取 Run, 运行 LangGraph 图, 心跳, 批量上报事件, 调用工具网关, 写 checkpoint, 完成 Run。	cd_kernel (只能访问 lg schema)
mock-llm	:7330	OpenAI 兼容 /v1/chat/completions, 按场景返回。	无

3.2 写入边界（整份设计的地基）

- 1 PostgreSQL 是唯一事实来源。Redis 丢任何消息都只影响实时推送，SSE 客户端重连后从数据库补齐。
- 2 public 表只有 Go 写。内核想改变任何业务状态（事件、工具调用、完成），都经内部 API 提出，由 Go 校验租约后落库。
- 3 内核唯一直接写的是 checkpoint，且每次写入在同一事务里经触发器校验租约（C3）。
- 4 所有带租约的写入都是条件 UPDATE: `WHERE id=$run AND attempt=$attempt AND lease_owner=$worker AND status='running'`。0 行受影响即 409 lease_lost。

5 时间只用数据库的 `now()`。任何进程都不拿本地时钟判断租约是否过期；过期由 reaper 统一裁决。

3.3 一次 Run 的完整路径

1. POST `/v1/sessions/{sid}/runs` api: 校验会话无活跃 Run → INSERT `runs(status=queued)` → 事件 `run.queued`
2. 内核 POST `/internal/runs/claim` api: SKIP LOCKED 领取 → `status=running, attempt+1, lease 30s` → 事件 `run.started`
3. 内核读 `checkpoint(thread_id=run_id)` 有 → 事件 `run.resumed`, 从断点继续; 无 → 从 `system+user` 消息开始
4. 循环: agent 节点 (调 LLM) → tools 节点 (逐个 POST `/internal/runs/{id}/tool-calls`)
 每个节点结束 LangGraph 写 `checkpoint` (触发器校验租约)
 每 10s 心跳: 续租 30s; 响应里带 `cancel_requested`
 事件每 `≤500ms` 或 `≤20` 条批量 POST `/internal/runs/{id}/events`
5. submit_result 成功 → POST `/internal/runs/{id}/complete {status: succeeded, result}`
6. reaper: 若 `lease_expires_at < now()` → `attempt < max_attempts` 则回到 `queued`, 否则 `failed`

4 技术选型

部分	选择	理由
Go	Go 1.25+, 标准库 <code>net/http</code> (1.22 起的方法+路径模式路由)	路由够用、SSE 直接写 <code>http.Flusher</code> ; 少一个框架就少一层要学的东西。需要 <code>os.Root</code> 的 <code>MkdirAll / Rename</code> (1.25) 。
Go 数据库	pgx v5 (<code>pgxpool</code>) , 手写 SQL	这个项目的难点就在 SQL 语义上, 不要 ORM 挡在中间。
迁移	goose v3, SQL 文件 <code>//go:embed</code>	和旧 P0 一致, 简单可靠。
Redis 客户端	go-redis v9	只用 <code>PUBLISH / SUBSCRIBE</code> 。
Docker	Docker Engine Go SDK (moby client) , 仅 <code>internal/sandboxd</code> 使用	archtest 强制。
日志/配置	<code>log/slog</code> JSON; 环境变量 (附 B)	零依赖。
Python	3.12, uv 管理	—
内核	<code>langgraph</code> 、 <code>langgraph-checkpoint-postgres</code> 、 <code>psycopg[binary]</code> 3、 <code>langchain-openai</code> 、 <code>httpx</code>	用 <code>ChatOpenAI(base_url=...)</code> 对接 mock 或任意 OpenAI 兼容端点。
Mock LLM	FastAPI + uvicorn + PyYAML	—
测试	Go <code>testing</code> ; <code>pytest</code> + <code>pytest-asyncio</code> ; 端到端用 <code>pytest</code> 驱动真实进程	测试库直接连 <code>compose</code> 的 PostgreSQL, 每个测试建临时数据库。
基础设施	<code>docker compose</code> : <code>postgres:16</code> 、 <code>redis:7</code> 、 <code>mock-llm</code>	Go 与 Python 进程在宿主机直接跑 (见 ADR-06) , 方便 <code>kill -9</code> 和挂载路径一致。

版本锁定

A1 时把实际安装的版本写进 `go.mod` 、 `uv.lock` 和 `README`。报告里不锁死小版本, 因为 `LangGraph` 的 `checkpoint` 表结构会随版本变化; C3 的触发器只依赖三张表都有 `thread_id` 列这一稳定事实, A1 之后不要随意升级 `langgraph-checkpoint-postgres` 。

5 全局约定

5.1 仓库结构（最终形态）

```

cakerdesk/
├── go/
│   ├── cmd/cakerdesk/main.go      # module github.com/<you>/cakerdesk
│   ├── migrations/                # 子命令 migrate; -role=api|reaper|sandboxd
│   │   └── 0001_agents.sql ... 0007_run_skills.sql (embed)
│   └── internal/
│       ├── config/ apperr/ db/ httpx/    # 基础设施
│       ├── agents/                    # Agent、AgentVersion、配置校验
│       ├── sessions/                  # Session、工作区目录、文件上传下载
│       ├── runs/                      # 创建、领取、心跳、完成、取消、reaper
│       ├── events/                    # 追加（分配 seq）、发布、SSE
│       ├── gateway/                   # 工具网关：权限、幂等、执行语义、在飞调用表
│       ├── tools/                     # files.go bash.go skill.go submit.go
│       ├── skills/                     # SKILL.md 解析、目录哈希
│       ├── sandboxd/                   # 服务端（唯一 import Docker SDK）
│       ├── sandboxclient/              # api 调 sandboxd 的客户端
│       ├── kernelapi/                  # 内部 API 路由与鉴权
│       └── archtest/
├── kernel/                            # uv 项目 cd-kernel
│   ├── src/cd_kernel/
│   │   ├── worker.py client.py graph.py state.py llm.py setup.py
│   │   ├── heartbeat.py events.py
│   │   └── middleware/ retry.py budget.py loop_guard.py compaction.py
│   └── tests/
├── mockllm/ app.py scenarios/*.yaml tests/
├── sandbox/Dockerfile                  # cakerdesk-sandbox:dev
├── skills/csv-report/ SKILL.md scripts/aggregate.py
├── deploy/ compose.yaml postgres-init/00-roles.sql
├── tests/ e2e/ sabotage/patches/*.patch sabotage/run.sh
├── demo/ seed_csv.py demo.sh
├── var/ workspaces/ run/                # 运行时目录，gitignore
├── docs/ later.md interview.md
├── Makefile
└── README.md
  
```

5.2 ID、时间与命名

- 实体 ID 用 UUID (PostgreSQL 13+ 内置 `gen_random_uuid()`)，不需要 `pgcrypto` 扩展。
- `worker_id` 由内核启动时生成：`<hostname>-<pid>-<8位随机>`。每个内核进程一个，重启即换。
- `executor_id` 由 api 启动时生成，格式同上，用于判断工具调用的执行者是否还活着（D4）。
- 所有时间列 `timestamptz`；所有租约计算用 `now()`。
- 状态与枚举用 `text + CHECK`，不用 PostgreSQL enum 类型（增删值要迁移，学习成本高）。

5.3 错误响应格式

```

HTTP 4xx/5xx
{"error": {"code": "lease_lost", "message": "run 7cl... attempt 2 is no longer owned by worker w-..."}}
  
```

`internal/apperr` 定义 `type Error struct { Code string; Status int; Msg string }`，处理器只返回 `*apperr.Error` 或普通 `error`（普通 `error` 一律 500 `internal`，并记录日志）。全部错误码见附 A。

5.4 测试约定

- Go 测试用 `CD_TEST_DATABASE_URL` (指向 compose 的 postgres, 超级用户)。辅助函数 `dbtest.New(t)` 创建 `cd_test_<随机>` 数据库、跑全部迁移、`t.Cleanup` 删除。每个测试拿到的是干净库。
- 测试名是契约: 破坏测试 (第 16 章) 按名字运行, 改名要同步改 `tests/sabotage`。
- 需要真实进程的测试 (崩溃、僵尸、容器) 放在 `tests/e2e`, 用 `pytest` 起子进程, `make e2e` 运行。
- 禁止 `time.Sleep` 等状态: 用 “轮询直到条件成立或超时” 的辅助函数 `eventually(t, 10*time.Second, func() bool)`。
- 测试里可以把租约缩短: `CD_LEASE_SECONDS=3`、`CD_HEARTBEAT_SECONDS=1`、`CD_REAPER_INTERVAL_MS=300`。

5.5 防桩规则

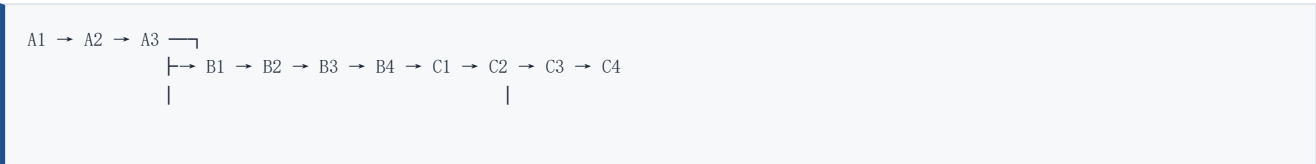
- `make lint-stubs` 在 CI 和 `make test` 中运行, 命中即失败。
- 不允许 “先返回固定值、以后再实现” 的处理器。某一步用不到的接口, 就不注册路由。
- 每个工具在注册时必须能真实执行; 不存在 “占位工具”。工具注册表随步骤增长: B3 只有 `submit_result`, D1 加文件工具, D3 加 `bash`, E1 加 `load_skill`。

6 阶段总览

19 个小步分 6 个阶段。每一步都结束在“能演示的状态”。下表的“演示”一栏就是这一步完成时你能给别人看的東西。

步	名称	新增能力	演示
A1	仓库与基础设施	Go/Python 项目、compose、Makefile、lint、archtest 骨架	<code>make up && make test</code> 全绿; <code>/healthz</code> 返回 ok
A2	数据库与 Agent 版本	迁移、三个 DB 角色、API Key、Agent/Version API	创建 Agent、发布 v1/v2; 改历史版本被数据库拒绝
A3	Mock LLM	YAML 场景、唯一 <code>tool_call_id</code> 、错误注入次数	<code>curl</code> 一个场景, 看到逐步返回的工具调用
B1	Session 与 Run 入队	<code>sessions/runs</code> 表、工作区目录、一个会话只有一个活跃 Run	第二个 Run 返回 409 <code>session_busy</code>
B2	Claim	内部 API、 <code>SKIP LOCKED</code> 长轮询、完成接口	两个并发 <code>claim</code> 各拿到不同 Run
B3	LangGraph 内核	worker 循环、图、 <code>submit_result</code> 、 <code>checkpoint</code>	一个 Run 从 <code>queued</code> 跑到 <code>succeeded</code> , 结果可查
B4	事件与 SSE	<code>run_events</code> 、 <code>seq</code> 、 <code>Pub/Sub</code> 、 <code>Last-Event-ID</code>	<code>curl -N</code> 实时看时间线; 断开重连不丢事件
C1	租约与心跳	心跳续租、 <code>fencing</code> 条件写、409 处理、自我隔离	手动改 <code>attempt</code> , 旧 worker 立刻停止
C2	Reaper 与续跑	过期回收、重新排队、从 <code>checkpoint</code> 续跑	<code>kill -9</code> 内核, 另一个内核接手续跑
C3	Checkpoint fencing	<code>fencing</code> 触发器、 <code>FOR SHARE</code> 、 <code>cd_kernel</code> 权限	<code>SIGSTOP</code> / <code>SIGCONT</code> 僵尸写 <code>checkpoint</code> 被拒
C4	取消与超时	<code>cancel_requested_at</code> 、 <code>deadline</code> 、 <code>max_attempts</code>	运行中取消, 在下一个节点边界停止
D1	网关持久化与文件工具	<code>tool_executions</code> 、幂等重放、 <code>os.Root</code> 文件工具、文件上传	同一 <code>tool_call_id</code> 调两次只执行一次
D2	执行语义	<code>started</code> 先提交、 <code>executor_id</code> 、恢复规则、 <code>interrupted</code> 、请求取消	两个网关实例模拟 <code>api</code> 重启: 非幂等调用报告 <code>interrupted</code>
D3	sandboxd	加固会话容器、 <code>exec</code> 、超时/断开杀进程组	容器内联网失败、写根目录失败、 <code>sleep</code> & 子进程被杀
D4	bash 与崩溃恢复	<code>bash</code> 工具、取消链路、端到端 <code>kill -9</code>	<code>bash</code> 中途杀内核: <code>bash</code> 报 <code>interrupted</code> , <code>read_file</code> 重放
E1	Skill 与权限	<code>SKILL.md</code> 、 <code>load_skill</code> 、有效工具集合、 <code>tool_not_allowed</code>	未加载 Skill 就调 <code>bash</code> 被拒, 加载后成功
E2	重试/预算/循环保护	429/5xx 重试、预算计数进状态、重复调用告警与失败	注入两次 429 后成功; 死循环场景第 5 次失败
E3	上下文压缩	摘要 + 保留最近消息、不拆工具对	长任务出现 <code>context.compacted</code> 后仍正确完成
F1	破坏测试与演示	全部破坏补丁、 <code>demo.sh</code> 、 <code>README</code> 、面试文档	完整演示脚本 (第 17 章)

6.1 依赖关系



|
A3 被 B3 起的所有步骤使用（全部测试走 mock）
↳ D1 → D2 → D3 → D4 → E1 → E2 → E3 → F1

为什么是这个顺序：

- **事件 (B4) 放在可靠性 (C) 之前**：之后每一步的验证都靠看时间线，先有眼睛再做手术。
- **C 在 D 之前**：工具执行语义依赖 attempt 与 fencing；没有租约就谈不上“恢复时怎样处理未完成的工具调用”。
- **执行语义 (D2) 在沙箱 (D3) 之前**：恢复规则用 `_test.go` 里的测试替身工具证明，生产注册表里从不出假工具；D4 再用真实的 bash 做端到端验证。
- **Skill 权限 (E1) 在 bash 之后**：权限约束最有说服力的对象就是 bash。

7 阶段 A：地基

STAGE A · A1-A3

让仓库能跑、能测、能拒绝桩

这一阶段没有任何 Agent 逻辑。结束时你有：一键起基础设施、一键跑全部测试、数据库迁移与角色、一个带版本的 Agent 资源，以及一个可预测的假模型。

A1 仓库与本地基础设施

依赖：无 · tag: step-A1

目标： `make up` 起 PostgreSQL 与 Redis；`make test` 跑 Go 与 Python 测试；`make lint` 包含格式检查、lint-stubs 和 `archtest`；`cakdesk -role=api` 能返回 `/healthz`。

学习

- Go module 布局、`internal/` 的可见性规则。
- `net/http` 1.22 路由模式：`mux.HandleFunc("GET /healthz", h)`、`r.PathValue("id")`。
- UV：`uv init --package kernel`、`uv add`、`uv run pytest`。
- `docker compose` 的 `healthcheck` 与 `depends_on: condition: service_healthy`。

产出

文件	要点
<code>go/go.mod</code> 、 <code>go/cmd/cakdesk/main.go</code>	解析 <code>-role</code> ； <code>api</code> 启动公共 <code>mux (:7310)</code> ；其他角色此时报错 <code>unknown role</code> 并退出码 2（不是桩：未实现的角色不存在）。优雅退出： <code>signal.NotifyContext(SIGINT, SIGTERM)</code> + <code>srv.Shutdown</code> 。
<code>go/internal/config</code>	<code>Load()</code> (<code>Config, error</code>) 从环境变量读，缺必填项报错；附 B 的每个配置项在出现它的步骤加入。
<code>go/internal/httpx</code>	<code>WriteJSON</code> 、 <code>WriteError</code> (5.3 格式)、 <code>DecodeJSON</code> (<code>DisallowUnknownFields</code> ， <code>body</code> 上限 1 MiB)、请求日志中间件 (<code>method</code> 、 <code>path</code> 、 <code>status</code> 、耗时)。
<code>go/internal/apperr</code>	见 5.3。
<code>go/internal/archtest/arch_test.go</code>	运行 <code>go list -deps -json ./...</code> ，规则以表驱动： <code>{pkgPrefix, forbiddenImport}</code> 。A1 先放一条： <code>internal/...</code> 除 <code>sandboxd</code> 外不得 <code>import</code> <code>github.com/docker/</code> 、 <code>github.com/moby/</code> 。
<code>kernel/</code>	<code>uv init --package</code> ，入口 <code>cd-kernel = cd_kernel.worker:main</code> ；A1 的 <code>main</code> 只支持 <code>--version</code> 。 <code>ruff</code> 配置。
<code>deploy/compose.yaml</code>	<code>postgres:16</code> (7340:5432, <code>volume pgdata</code> ，挂载 <code>postgres-init/</code>)、 <code>redis:7</code> (7341:6379, 不持久化)。两者都有 <code>healthcheck</code> 。
<code>Makefile</code>	<code>up down test lint lint-stubs arch e2e run-api</code> 。 <code>test = go test ./... + uv run pytest</code> (<code>kernel</code> 、 <code>mockllm</code>)。
<code>scripts/lint-stubs.sh</code>	<code>rg -n 'TODO FIXME XXX NotImplemented unimplemented raise NotImplementedError'</code> <code>go kernel mockllm --glob '!*/testdata/*'</code> ，有输出即退出 1。
<code>.gitignore</code> 、 <code>README.md</code>	忽略 <code>var/</code> 、 <code>.venv/</code> 、 <code>.env</code> 。 <code>README</code> 写：一句话定位、如何启动、进度清单 (19 步)。

测试

- `TestHealthz` : `httptest` 请求 `GET /healthz` → 200 `{"status": "ok"}`。
- `TestArchRules` : 规则表通过; 另加一个自测: 对一个构造的依赖列表断言规则能检出违规。
- `kernel/tests/test_version.py` : `cd-kernel --version` 输出版本号。

完成标准

- 全新克隆后 `make up` && `make test` && `make lint` 全绿。
- 在任意 Go 文件写一行 `// TODO`, `make lint` 失败。
- README 有启动步骤和进度清单。

本步不做

- 数据库连接 (A2)、mock-llm 容器 (A3)、任何业务路由。

坑

`lint-stubs` 不要扫 `docs/` (报告里会出现 `TODO` 字样)。`archtest` 用 `go list` 而不是解析源码, 才能覆盖间接依赖。

A2 数据库、迁移与 Agent 版本

依赖: A1 · 迁移: 0001_agents.sql · tag: step-A2

目标: 有三个数据库角色、可重复执行的迁移、静态 API Key 鉴权, 以及 Agent/AgentVersion 的创建与发布接口; 历史版本在数据库层面不可修改。

学习

- PostgreSQL 角色、schema、GRANT、ALTER DEFAULT PRIVILEGES (为什么以后新建的表会自动授权)。
- pgx v5: `pgxpool.New`、`pool.BeginFunc` / `pgx.BeginFunc`、`pgx.CollectRows`、错误码 `*pgconn.PgError`。
- goose 的 Up/Down 与 `embed.FS`。
- “不可变版本 + 当前版本指针” 这个模式为什么让运行可复现。

产出: 角色 (集群级, compose 初始化时由超级用户执行)

```
-- deploy/postgres-init/00-roles.sql
CREATE ROLE cd_migrate LOGIN PASSWORD 'cd_migrate'; -- 表的属主, 只用于迁移和 kernel setup
CREATE ROLE cd_app LOGIN PASSWORD 'cd_app'; -- api 与 reaper
CREATE ROLE cd_kernel LOGIN PASSWORD 'cd_kernel'; -- 内核, 只能访问 lg schema
ALTER ROLE cd_kernel SET search_path = lg;
CREATE DATABASE cakerdesk OWNER cd_migrate;
```

产出: 0001_agents.sql (以 cd_migrate 执行)

```
-- +goose Up
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
GRANT USAGE ON SCHEMA public TO cd_app;
ALTER DEFAULT PRIVILEGES FOR ROLE cd_migrate IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO cd_app;

CREATE SCHEMA lg AUTHORIZATION cd_migrate;
GRANT USAGE ON SCHEMA lg TO cd_kernel;
```

```

ALTER DEFAULT PRIVILEGES FOR ROLE cd_migrate IN SCHEMA lg
  GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO cd_kernel;

CREATE TABLE agents (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name        text NOT NULL UNIQUE CHECK (name ~ '^[a-z0-9]{1,62}$'),
  current_version int,
  created_at   timestampz NOT NULL DEFAULT now(),
  updated_at   timestampz NOT NULL DEFAULT now()
);

CREATE TABLE agent_versions (
  agent_id    uuid NOT NULL REFERENCES agents(id),
  version      int NOT NULL CHECK (version >= 1),
  config       jsonb NOT NULL,
  config_hash  text NOT NULL,
  created_at   timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (agent_id, version)
);
REVOKE UPDATE, DELETE ON agent_versions FROM cd_app;  -- 数据库层面的不可变

-- +goose Down
DROP TABLE agent_versions; DROP TABLE agents; DROP SCHEMA lg;

```

产出：AgentConfig (V1 最终形态，各字段在对应步骤开始生效)

```

{
  "model": "mock-1",           // 传给 LLM 的模型名
  "system_prompt": "You are a careful data analyst...",
  "tools": ["load_skill", "list_files", "read_file", "write_file", "bash", "submit_result"],
  "skills": ["csv-report"],    // E1 起校验
  "limits": {
    "max_llm_calls": 40,       // E2
    "max_tool_calls": 80,      // E2
    "max_total_tokens": 200000, // E2
    "max_duration_s": 1800,    // C4
    "max_attempts": 3          // C2
  },
  "context": { "compact_threshold_tokens": 12000, "keep_last_messages": 8 } // E3
}

```

校验规则	细节
tools	非空、无重复；每个名字必须在 <code>tools.Specs</code> 注册表里；必须包含 <code>submit_result</code> 。注册表随步骤增长 (5.5)，A2 时只有 <code>submit_result</code> 的规格（名字、描述、JSON Schema、Idempotent），执行在 B3 加入。
limits	<code>max_llm_calls</code> 1-500； <code>max_tool_calls</code> 1-1000； <code>max_total_tokens</code> 1000-5,000,000； <code>max_duration_s</code> 10-86400； <code>max_attempts</code> 1-10。缺省取上面的默认值。
context	<code>compact_threshold_tokens</code> 2000-200000； <code>keep_last_messages</code> 2-50。
system_prompt	≤ 20000 字符。
未知字段	拒绝（ <code>DisallowUnknownFields</code> ），返回 400 <code>invalid_config</code> ， <code>message</code> 指出字段路径。

产出：接口

接口	行为
POST <code>/v1/agents</code> <code>{"name"}</code>	201 <code>{id,name,current_version:null}</code> ；重名 409 <code>agent_exists</code> 。
GET <code>/v1/agents</code> 、GET <code>/v1/agents/{id}</code>	列表按 <code>created_at</code> 倒序。

接口	行为
POST /v1/agents/{id}/versions {"config"}	事务内：SELECT ... FROM agents WHERE id=\$1 FOR UPDATE；规范化配置（填默认值后 json.Marshal 结构体，字段顺序固定）并算 sha256；若与当前版本哈希相同，200 返回当前版本（幂等发布）；否则插入 version = coalesce(max,0)+1，更新 current_version，201。
GET /v1/agents/{id}/versions/{version}	返回规范化后的配置。

鉴权：中间件比较 Authorization: Bearer <CD_API_KEY>，用 subtle.ConstantTimeCompare；/healthz 豁免。/healthz 此步加上 pool.Ping。

命令：cakerdesk migrate up|down|status，使用 CD_MIGRATE_DATABASE_URL（cd_migrate）。api 使用 CD_DATABASE_URL（cd_app）。

测试

- TestAgentVersionImmutable：以 cd_app 连接执行 UPDATE agent_versions ...，断言 SQLSTATE 42501（权限不足）。
- TestPublishIncrementsVersion、TestPublishSameConfigIsIdempotent。
- TestPublishConcurrent：10 个 goroutine 同时发布不同配置，版本号恰好是 1..10、无冲突报错。
- TestConfigValidation：表驱动，每条规则至少一个反例。
- TestAuthRequired：无/错 Key → 401 unauthorized。
- dbtest.New 本身：断言新库里 cd_app 对新建测试表自动有 SELECT（验证默认权限生效）。

完成标准

- make migrate 在空库上成功，再执行一次无副作用。
- curl 能创建 Agent、发布两个版本；用 psql 以 cd_app 改 v1 被拒。
- 所有测试通过，且每个测试用独立数据库。

本步不做

- Skill 校验 (E1)；Agent 删除/重命名 (V1 不做)；分页（列表上限 200 条即可）。

坑

ALTER DEFAULT PRIVILEGES FOR ROLE cd_migrate 只对 cd_migrate 之后创建的表生效，所以迁移必须以 cd_migrate 身份运行，不能用 postgres 超级用户跑（否则表属主是 postgres，cd_app 没权限）。这正是旧 P0 用 GRANT ... ON ALL TABLES 留下的隐患。

A3 Mock LLM

依赖：A1 · tag: step-A3

目标：一个 OpenAI 兼容的假模型，按 YAML 场景逐步返回工具调用，**无状态地**从请求内容推断当前步骤，支持按次数的错误注入和延迟。之后所有测试和演示都依赖它。

学习

- OpenAI Chat Completions 的 tool calling 格式：tool_calls[].id、function.arguments 是 JSON 字符串、role: tool 消息用 tool_call_id 回应。
- 为什么假模型必须无状态：内核崩溃后会重发同一请求，服务端游标会错位。

场景格式

```
# mockllm/scenarios/csv-report.yaml
name: csv-report
steps:
  - tool_calls:
    - {name: load_skill, args: {name: csv-report}}
  - tool_calls:
    - {name: list_files, args: {path: data}}
  - tool_calls:
    - {name: read_file, args: {path: data/sales_01.csv, limit: 5}}
    - {name: read_file, args: {path: data/sales_02.csv, limit: 5}}
    - {name: bash, args: {command: "python3 /skills/csv-report/scripts/aggregate.py data out/summary.csv && sleep 15",
timeout_s: 60}}
  - repeat: 3
    tool_calls:
    - {name: list_files, args: {path: out}}
  - content: "Done."
  - tool_calls:
    - {name: submit_result, args: {summary: "Aggregated 20 files", data: {rows: 20}}}
errors:
  - {step: 1, status: 429, times: 2}
delays:
  - {step: 2, ms: 1500}
```

协议规则

规则	细节
场景选择	第一条 user 消息中的 [[scenario:NAME]]。找不到 → 400 {"error":{"message":"unknown scenario"}}。
步骤推断	从后往前找最后一条带 tool_calls 的 assistant 消息，用正则 ^call_(\d+)(\d+)[a-z0-9]{6}\$ 解析出步骤 s，本次返回步骤 s+1；找不到则返回步骤 0。超过最后一步则一直返回最后一步。
tool_call_id	call_{step}_{k}_{6位随机}。同一 Run 内唯一（步骤递增），跨请求不重复；随机后缀模拟真实模型，防止实现依赖 id 可预测。
arguments	json.dumps(args, ensure_ascii=False)。
错误注入	计数键 (request.user, step)；user 字段由内核填 run_id。未超过 times 时返回对应 HTTP 状态和 OpenAI 风格错误体；429 附 retry-after: 0。
摘要模式	system 消息含 [[cd:summarize]] 时，不推进步骤，返回 content = "SUMMARY: {n} messages condensed." (E3 用)。
usage	prompt_tokens = 所有消息字符数 // 4，completion_tokens = 输出字符数 // 4。
调试接口	GET /mock/requests?user= 返回最近 500 条请求摘要（步骤、是否注入错误、工具列表）；POST /mock/reset 清空计数与日志。

场景文件在启动时全部加载并校验（纯文本步骤只能在最后、repeat ≥ 1、工具名非空），校验失败则启动失败。compose 增加 mock-llm 服务 (7330)。

测试 (mockllm/tests)

- `test_ids_unique_and_parseable` : 跑完整场景, 所有 id 唯一且匹配正则。
- `test_arguments_are_json` : `json.loads` 每个 arguments 成功。
- `test_step_from_history` : 构造只保留最后一对 AI/Tool 消息的历史 (模拟压缩), 步骤仍正确。
- `test_replay_same_request_same_step` : 同一请求发两次, 返回同一步 (工具名、参数一致, id 不同)。
- `test_error_times` : 前两次 429, 第三次成功; 换一个 user 重新计数。
- `test_repeat_expansion`、`test_summarize_mode`、`test_invalid_scenario_rejected_at_startup`。

完成标准

- 用 openai Python SDK (`base_url=http://127.0.0.1:7330/v1`) 能解析 mock 的每个响应。
- 旧 P0 的三个 bug (id 重复、arguments 非 JSON、错误无次数) 各有一个测试覆盖。

本步不做

- 流式响应、embedding 接口、按内容分支的场景。

8 阶段 B：最小运行闭环

STAGE B · B1-B4

一个 Run 从入队到成功，全程可观察

结束时：创建 Run → 内核领取 → LangGraph 跑图调用 mock → submit_result → succeeded；整个过程通过 SSE 实时可见。此阶段 **不回收**，租约字段存在但不续期、不回收。

B1 Session 与 Run 入队

依赖：A2 · 迁移：0002_sessions_runs.sql · tag: step-B1

目标：能创建 Session（带工作区目录）和 Run（绑定当前版本并快照配置）；一个 Session 同时最多一个活跃 Run，由数据库保证。

学习

- 部分唯一索引（partial unique index）：用约束而不是“先查后写”防并发。
- CHECK 约束表达状态不变量。
- SQLSTATE 23505 与约束名的映射。

产出：0002_sessions_runs.sql

```
CREATE TABLE sessions (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_id    uuid NOT NULL REFERENCES agents(id),
  title       text NOT NULL DEFAULT '',
  created_at  timestampz NOT NULL DEFAULT now()
);

CREATE TABLE runs (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id  uuid NOT NULL REFERENCES sessions(id),
  agent_id    uuid NOT NULL,
  agent_version int NOT NULL,
  config       jsonb NOT NULL,           -- 创建时的配置快照
  input       text NOT NULL CHECK (length(input) BETWEEN 1 AND 20000),
  status      text NOT NULL DEFAULT 'queued'
  CHECK (status IN ('queued', 'running', 'succeeded', 'failed', 'cancelled')),
  attempt     int NOT NULL DEFAULT 0,   -- fencing token, 每次领取 +1
  max_attempts int NOT NULL,
  lease_owner  text,
  lease_expires_at timestampz,
  result       jsonb,
  error_code   text,
  error_message text,
  created_at  timestampz NOT NULL DEFAULT now(),
  started_at  timestampz,
  finished_at timestampz,
  updated_at  timestampz NOT NULL DEFAULT now(),
  FOREIGN KEY (agent_id, agent_version) REFERENCES agent_versions(agent_id, version),
  CHECK ((status = 'running') = (lease_owner IS NOT NULL AND lease_expires_at IS NOT NULL)),
  CHECK ((status IN ('succeeded', 'failed', 'cancelled')) = (finished_at IS NOT NULL))
);

CREATE UNIQUE INDEX runs_one_active_per_session ON runs(session_id)
  WHERE status IN ('queued', 'running');
```

```
CREATE INDEX runs_queued_fifo ON runs(created_at) WHERE status = 'queued';
CREATE INDEX runs_running_lease ON runs(lease_expires_at) WHERE status = 'running';
```

产出：接口

接口	行为
POST /v1/sessions { "agent_id", "title?" }	Agent 必须有 current_version, 否则 409 agent_unpublished。事务内插入后 os.MkdirAll(var/workspaces/<id>, 0o755), 目录失败则回滚。
GET /v1/sessions/{id}	含 active_run_id (查询活跃 Run)。
POST /v1/sessions/{id}/runs { "input" }	读 Agent 当前版本, 把配置写入 runs.config, max_attempts 取自配置。违反 runs_one_active_per_session → 409 session_busy (message 带当前活跃 run_id)。201 返回 Run。
GET /v1/runs/{id}、 GET /v1/sessions/{id}/runs	Run 视图: id, session_id, agent_version, status, attempt, result, error{code, message}, created_at, started_at, finished_at。 不返回 config (太大), 另有 GET /v1/runs/{id}/config。

测试

- TestCreateRunQueued: 状态 queued、attempt 0、config 等于当前版本配置。
- TestOneActiveRunPerSession: 20 个 goroutine 并发创建, 恰好 1 个 201、其余 409。
- TestRunSnapshotsConfig: 创建 Run 后发布 v2, Run 的 config 与 agent_version 不变。
- TestRunStatusInvariants: 直接 SQL 插入 status='running' 但 lease_owner 为空 → CHECK 失败。

完成标准

- curl 演示: 创建 Session、创建 Run、再创建一个 → 409 session_busy。
- var/workspaces/<session_id> 目录存在。

本步不做

- 取消 (C4)、文件上传 (D1)、Session 关闭与删除 (V1 不做)。

坑

不要写“先 SELECT 看有没有活跃 Run, 再 INSERT”。两个并发请求都会看到“没有”。约束才是唯一可靠的检查点, 代码只负责把 23505 翻译成 409。

B2 Claim: 数据库即队列

依赖: B1 · tag: step-B2

目标: 内部 API 上线; worker 通过长轮询领取 Run, 多个 worker 并发领取互不冲突; worker 能以 fencing 条件完成 Run。

学习

- `SELECT ... FOR UPDATE SKIP LOCKED`：锁住选中的行，跳过别人已锁的行——这就是用 PostgreSQL 做工作队列的全部秘密。
- `CTE + UPDATE ... FROM ... RETURNING` 的原子领取。
- 长轮询：服务端持有请求直到有结果或超时，比客户端高频轮询省连接。
- fencing token 的直觉：attempt 每次领取 +1，旧持有者的写入因为 attempt 不匹配而失败。

产出：内部 API 骨架（internal/kernelapi）

- 独立 listener 127.0.0.1:7312，中间件校验 `Authorization: Bearer <CD_INTERNAL_TOKEN>`。
- 所有带租约的请求体都包含 `{"worker_id", "attempt"}`，处理器组装成 `runs.Fence{RunID, Attempt, WorkerID}`。

产出：领取 SQL

```
WITH next AS (
  SELECT id FROM runs
  WHERE status = 'queued'
  ORDER BY created_at
  FOR UPDATE SKIP LOCKED
  LIMIT 1
)
UPDATE runs r
SET status = 'running',
    attempt = r.attempt + 1,
    lease_owner = $1,
    lease_expires_at = now() + make_interval(secs => $2),
    started_at = coalesce(r.started_at, now()),
    updated_at = now()
FROM next
WHERE r.id = next.id
RETURNING r.id, r.session_id, r.attempt, r.config, r.input;
```

接口	行为
<code>POST /internal/runs/claim</code> <code>{"worker_id", "wait_ms"}</code>	<code>wait_ms</code> 上限 20000。循环：执行领取 SQL；拿到则 200；拿不到则等 500ms 或请求 ctx 结束再试；到时返回 204。 200 响应： <pre>{run_id, session_id, attempt, input, config, tools: [(name, description, parameters)], lease_seconds, heartbeat_seconds}</pre> tools 从 Go 注册表按 <code>config.tools</code> 过滤，内核不自己维护工具定义。
<code>POST /internal/runs/{id}/complete</code> <code>{"worker_id", "attempt", "status", "result?", "error?": {"code", "message"}}</code>	<code>status ∈ {succeeded, failed}</code> （cancelled 在 C4 加入）。条件 UPDATE： <pre>... SET status=\$s, result=\$r, error_code=..., finished_at=now(), lease_owner=NULL, lease_expires_at=NULL WHERE id=\$id AND attempt=\$a AND lease_owner=\$w AND status='running' ; 0 行 → 409 lease_lost。</pre>

测试

- `TestClaimSkipLocked`：插 5 个 queued，10 个 goroutine 并发 claim (`wait_ms=0`)，恰好 5 个拿到且 `run_id` 互不相同。
- `TestClaimFIFO`：按创建顺序领取。
- `TestClaimLongPollWakes`：先发起 `wait_ms=5000` 的 claim，1 秒后创建 Run，claim 在 2 秒内返回该 Run。
- `TestClaimEmptyReturns204`。
- `TestCompleteRequiresFence`：错 attempt、错 worker、已完成的 Run 各返回 409。
- `TestInternalAPIRequiresToken`。

完成标准

- 用两个终端 curl claim，能看到各自拿到不同的 Run；用错误 attempt complete 得到 409。

本步不做

- 心跳与回收（C1、C2）：此时 Run 被领走后若 worker 不 complete，会永远停在 running——这是有意暴露的问题，C2 解决。
- LISTEN/NOTIFY 唤醒（写进 later.md，500ms 轮询在 V1 足够）。

坑

长轮询循环里每次领取都是一个独立的短事务，不要开一个长事务包住整个等待，否则会一直占着连接和快照。

pgxpool 的 MaxConns 至少设为“预期内核数 + 10”。

B3 LangGraph 内核跑通

依赖：B2、A3 · tag: step-B3

目标：Python 内核领取 Run，用 LangGraph 构建 agent/tools 循环，调用 mock 模型，通过网关执行 submit_result，并完成 Run；每一步的状态存入 PostgreSQL checkpoint。

学习

- LangGraph：StateGraph、add_messages reducer、条件边、compile(checkpointer=...)、thread_id。
- checkpoint 的粒度：**每个节点（super-step）结束时**写一次；节点执行到一半崩溃，恢复时整个节点重跑。这是 D1/D4 执行语义的根源。
- AsyncPostgresSaver 的连接要求：autocommit=True、row_factory=dict_row。
- ainvoke(None, config)：从已有 checkpoint 继续，而不是重新开始。

产出：checkpoint 表初始化

cd-kernel setup：以 CD_KERNEL_SETUP_DATABASE_URL（cd_migrate，options=-csearch_path=lg）连接，调用 AsyncPostgresSaver.setup()，在 lg schema 建表；表属主是 cd_migrate，默认权限自动授予 cd_kernel。make migrate = cakdesk migrate up + cd-kernel setup。

产出：内核结构

```
# state.py
class RunState(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]
    result: dict | None          # submit_result 成功后写入
    nudges: int                 # 模型不调工具时的提醒次数

# graph.py
def build_graph(ctx: RunContext) -> StateGraph:
    g = StateGraph(RunState)
    g.add_node("agent", agent_node)      # 调 LLM，返回 AIMessage
    g.add_node("tools", tools_node)     # 逐个调用网关，返回 ToolMessage 列表
    g.add_node("nudge", nudge_node)     # 追加 "You must call submit_result when done."
    g.add_edge(START, "agent")
    g.add_conditional_edges("agent", route_after_agent, ["tools", "nudge", END])
    g.add_conditional_edges("tools", lambda s: END if s["result"] else "agent", ["agent", END])
    g.add_edge("nudge", "agent")
```

```

    return g

def route_after_agent(s):
    last = s["messages"][-1]
    if last.tool_calls: return "tools"
    if s["nudges"] < 2: return "nudge"
    raise RunFailed("no_result", "model stopped without calling submit_result")

```

```

# worker.py (核心流程)
async def execute_run(api, claim, worker_id):
    ctx = RunContext(api=api, claim=claim, worker_id=worker_id)
    async with await psycpg.AsyncConnection.connect(
        KERNEL_DB_URL, autocommit=True, row_factory=dict_row) as conn:
        saver = AsyncPostgresSaver(conn)
        graph = build_graph(ctx).compile(checkpointer=saver)
        cfg = {"configurable": {"thread_id": claim.run_id}, "recursion_limit": 400}
        existing = await saver.aget_tuple(cfg)
        inp = None if existing else {"messages": [SystemMessage(claim.config.system_prompt),
                                                    HumanMessage(claim.input)],
                                     "result": None, "nudges": 0}

        try:
            final = await graph.ainvoke(inp, cfg)
            await api.complete(ctx.fence, "succeeded", result=final["result"])
        except RunFailed as e:
            await api.complete(ctx.fence, "failed", error={"code": e.code, "message": str(e)})

```

- **LLM**: ChatOpenAI(model=config.model, base_url=CD_LLM_BASE_URL, api_key=CD_LLM_API_KEY, max_retries=0, timeout=60).bind_tools(claim.tools).bind(user=run_id)。max_retries=0 是因为重试由 E2 的中间件负责并计入事件。
- **工具调用**: POST /internal/runs/{id}/tool-calls {worker_id, attempt, tool_call_id, name, args} → {status: "succeeded"|"failed", output: str, error_code?}。内核把结果包成 ToolMessage(content=output, tool_call_id=..., status="error" if failed)。一条 AImessage 的多个调用顺序执行（顺序对 D4 的重放演示很重要）。
- **B3 的网关**: 校验租约（SELECT ... FOR SHARE 带 fence 条件）、校验工具在 config.tools 中且已注册、按 JSON Schema 校验参数（用 santhosh-tekuri/jsonschema），然后执行。此时注册表只有 submit_result: 参数 {summary: string 1..4000, data?: object}，执行结果 "result accepted"。持久化在 D1 加入。
- 一个内核进程同时只跑一个 Run；要并行就多起几个进程。

测试

- tests/e2e/test_b3_happy_path.py：起 api、kernel、mock；场景 hello（第 0 步直接 submit_result）→ Run succeeded，result 等于场景参数。
- test_no_result_fails：场景只有纯文本步骤 → 两次提醒后 failed，error_code=no_result；mock 请求日志恰好 3 次。
- test_checkpoint_written：以 cd_migrate 查询 lg.checkpoints，存在 thread_id = run_id 的行。
- test_invalid_tool_args：场景以非法参数调 submit_result → ToolMessage 是错误，模型下一步正确调用后成功。
- Go: TestToolCallRejectsUnknownTool、TestToolCallRequiresFence。

完成标准

- make run-api、make run-kernel 后，curl 创建 Run，几秒内 GET /v1/runs/{id} 显示 succeeded 和 result。
- 内核进程保持运行并继续领取下一个 Run。

本步不做

- 心跳 (C1)：B3 的 Run 若超过 30 秒会“租约过期”，但此时没有 reaper，没有后果。测试场景都在几秒内结束。
- 事件 (B4)、除 submit_result 外的工具 (D1 起)。

坑

LangGraph 的条件边函数里抛异常会让 `ainvoke` 抛出，这正是 `no_result` 的实现方式；但要确认异常类型没被 LangGraph 包装（捕获时用 `isinstance` 检查 `__cause__` 链，或者改为路由到一个 `fail` 节点再抛）。

B4 事件流与 SSE

依赖：B3 · 迁移：0003_run_events.sql · tag: step-B4

目标：Run 的每个关键动作都成为持久化事件，按 Run 单调递增 seq；客户端用 SSE 实时订阅，断线后用 Last-Event-ID 续传不丢不重。

学习

- 在行锁下分配序号：UPDATE runs SET last_seq = last_seq + n ... RETURNING last_seq，天然串行、无空洞。
- “先订阅、再回放、按 seq 去重”：消除回放与实时之间的竞态窗口。
- SSE 协议：id:、event:、data:、空行结束；浏览器/客户端重连时自动带 Last-Event-ID。
- Redis Pub/Sub 是“尽力而为”的：订阅者不在线就丢。所以它只能加速，不能作为事实来源。

产出：0003_run_events.sql

```
ALTER TABLE runs ADD COLUMN last_seq bigint NOT NULL DEFAULT 0;

CREATE TABLE run_events (
  run_id    uuid    NOT NULL REFERENCES runs(id) ON DELETE CASCADE,
  seq       bigint  NOT NULL,
  type      text    NOT NULL,
  attempt   int     NOT NULL,
  payload   jsonb   NOT NULL DEFAULT '{}',
  created_at timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (run_id, seq)
);
```

产出：追加事件 (internal/events)

```
// 在调用方的事务里执行；fence 为 nil 表示系统事件 (api/reaper 自己产生)
func Append(ctx context.Context, tx pgx.Tx, runID uuid.UUID, fence *runs.Fence,
  evs []New) ([]Event, error)

-- 带 fence:
UPDATE runs SET last_seq = last_seq + $2, updated_at = now()
WHERE id = $1 AND attempt = $3 AND lease_owner = $4 AND status = 'running'
RETURNING last_seq, attempt;
-- 无 fence: 去掉后三个条件
-- 0 行 (带 fence) → apperr lease_lost
-- 然后一次 INSERT ... SELECT FROM unnest(...) 写入 seq = last_seq-n+1 ... last_seq
```

发布：事务提交之后，对每个事件 PUBLISH run:<run_id> <event json>。发布失败只记日志。实现方式：服务层方法返回 []Event，处理器在 commit 成功后调用 pub.Publish(evs)。

事件类型 (V1 全集)

类型	产生方	payload	引入
run.queued	api	{agent_version}	B4

类型	产生方	payload	引入
run.started	api (claim 事务内)	{attempt, worker_id}	B4
run.resumed	kernel	{attempt, checkpoint_id, messages}	C2
message.assistant	kernel	{content, tool_calls:[{id,name,args}], usage}	B4
tool.started / tool.finished	api (网关事务内)	{tool_call_id, name, args} / {tool_call_id, status, replayed, exec_count, duration_ms, output_preview}	B4 / D1
skill.loaded	api	{name, hash, allowed_tools}	E1
llm.retry	kernel	{status, attempt_no, backoff_ms}	E2
guard.warning	kernel	{kind: "loop" "budget", detail}	E2
context.compacted	kernel	{removed, kept, summary_chars}	E3
run.lease_expired	reaper	{attempt, worker_id, next: "queued" "failed"}	C2
run.cancel_requested	api	{}	C4
run.succeeded / run.failed / run.cancelled	api/reaper	{result} / {code, message} / {}	B4 / C4

内核事件接口：POST /internal/runs/{id}/events {worker_id, attempt, events:[{type, payload}]}，每批 ≤100 条、每条 payload ≤64 KiB，type 必须在“kernel 可发类型”白名单内（上表产生方为 kernel 的那些）。内核侧 EventBuffer：每 500ms 或攒满 20 条刷一次，complete 前强制刷新。内核崩溃时缓冲区里未刷出的事件丢失——可以接受，因为执行状态在 checkpoint 里，事件只是观察窗口。

产出：公共接口

接口	行为
GET /v1/runs/{id}/events? after_seq=0&limit=500	JSON 列表，按 seq 升序。
GET /v1/runs/{id}/events/stream	SSE，算法如下。

```

1. sub := rdb.Subscribe(ctx, "run:"+id); 等待订阅确认 (sub.Receive 返回 *redis.Subscription)
2. last := Last-Event-ID 头 或 ?after_seq 或 0
3. 回放: SELECT ... WHERE run_id=$1 AND seq > last ORDER BY seq → 逐条写出, last = seq
4. 若已写出终态事件 (run.succeeded/failed/cancelled) → 结束
5. 循环 select:
    msg ← sub.Channel(): ev.seq <= last → 丢弃 (重复)
                        ev.seq > last+1 → 有缺口, 回到数据库补 seq > last 的全部事件
                        否则写出: 终态 → 结束
    每 15s: 写 ": ping\n\n"
    Redis 订阅出错: 降级为每 1s 查一次数据库, 直到连接恢复或客户端断开
写出格式: id: {seq}\nevent: {type}\ndata: {json}\n\n  每条后 Flush()

```

测试

- TestSeqMonotonicConcurrent：50 个 goroutine 各追加 1 条系统事件，seq 恰好 1..50。
- TestAppendFencedRejectsStaleAttempt：attempt 不匹配 → lease_lost，且 last_seq 不变。
- TestSSEReplayThenLive：先写 3 条，连接 SSE，再写 2 条，客户端按序收到 1..5。
- TestSSELastEventID：带 Last-Event-ID: 3 连接，只收到 4 起。
- TestSSEGapFill：直接 INSERT seq 4、只 PUBLISH seq 5，客户端仍收到 4 和 5。

- `TestSSEClosesOnTerminal`、`TestSSEWorksWithoutRedis` (Redis 地址指向关闭的端口, 事件靠轮询到达)。
- e2e: happy path 的事件序列恰为 `run.queued`, `run.started`, `message.assistant`, `tool.started`, `tool.finished`, `run.succeeded`。

完成标准

- `curl -N -H "Authorization: Bearer ..."` `.../events/stream` 能实时看到一个 Run 的全过程, 并在终态后自动断开。
- 中途 Ctrl-C 后带 `Last-Event-ID` 重连, 不丢不重。

本步不做

- token 级流式、事件保留期清理、跨 Run 的全局事件流。

坑

顺序一定是先订阅再回放。反过来的话, 回放查询结束到订阅生效之间产生的事件会永远丢失。另外 `run.started` 必须和领取在**同一个事务**里追加, 否则会出现“状态是 running 但没有 started 事件”的不一致。

9 阶段 C：运行可靠性

STAGE C · C1-C4

进程随时会死，Run 不能死；旧进程随时会醒，写入不能生效

结束时：内核被 `kill -9` 后 Run 由另一个内核从 checkpoint 续跑；被暂停后恢复的僵尸内核，其心跳、事件、工具调用、完成、全部被拒；取消和超时都可靠生效。

C1 租约、心跳与 fencing token

依赖：B4 · tag: step-C1

目标：内核每 10 秒续租一次；任何带租约的请求一旦返回 `409 lease_lost`，内核立即停止该 Run 的一切动作且不调用 complete；API 不可达时内核自我隔离。

学习

- 租约 (lease) = 有期限的锁；心跳 = 续期。为什么不用“永久锁 + 进程退出时释放”：`kill -9` 不会执行任何清理代码。
- Martin Kleppmann 的 fencing token 论证：进程暂停（GC、SIGSTOP、虚拟机迁移）后醒会以为自己还持有锁，唯一可靠的办法是让存储端拒绝旧 token。
- 为什么租约是否过期只由数据库裁决，而内核只用本地单调时钟做“保守的自我停止”。

产出

接口 / 模块	行为
POST /internal/runs/{id}/heartbeat { "worker_id", "attempt" }	<pre>UPDATE runs SET lease_expires_at = now() + make_interval(secs => \$4), updated_at = now() WHERE id = \$1 AND attempt = \$2 AND lease_owner = \$3 AND status = 'running' RETURNING lease_expires_at;</pre> 0 行 → <code>409 lease_lost</code> ; 200 {lease_expires_at} (C4 加 <code>cancel_requested</code>) 。
runs.CheckFence(ctx, tx, f)	<pre>SELECT 1 FROM runs WHERE id=\$1 AND attempt=\$2 AND lease_owner=\$3 AND status='running' FOR SHARE 。用于“先检查再做别的写入”的场景（工具网关）。 FOR SHARE 让 reaper 在本事务结束前改不了这一行。</pre>
内核 heartbeat.py	每个 Run 一个 <code>asyncio</code> 任务，间隔 <code>heartbeat_seconds</code> 。 <code>409</code> → 设 <code>ctx.lease_lost</code> , <code>main_task.cancel()</code> 。 网络错误/5xx → 记录，下一轮重试。 自我隔离：若距上次成功心跳 (<code>time.monotonic()</code>) $\geq$ <code>lease_seconds</code> , 视同 <code>lease_lost</code> 。
内核 client.py	任何接口返回 <code>409 lease_lost</code> 都抛 <code>LeaseLost</code> 。<code>execute_run</code> 捕获 <code>LeaseLost</code> (以及由它引起的 <code>CancelledError</code>) : 记录日志、关闭 DB 连接、不调用 complete、回到领取循环。

至此 fencing 覆盖：heartbeat、events、tool-calls、complete。只剩 checkpoint (C3) 。

测试

- `TestHeartbeatExtendsLease` : 两次心跳之间 `lease_expires_at` 严格增大。

- `TestHeartbeatStaleAttempt409`：用 `attempt-1` 心跳 → 409，且该行 `lease_expires_at` 不变。
- `TestHeartbeatWrongWorker409`。
- `e2e test_lease_lost_stops_kernel`：场景第 1 步延迟 8s；内核进入延迟后，用 SQL `UPDATE runs SET attempt = attempt + 1, lease_owner = 'intruder'` 模拟被接管；断言 $\leq$ 心跳间隔 + 1s 内内核日志出现 `lease_lost`，之后 mock 请求日志不再增长，Run 未被 complete。
- `e2e test_self_fence_when_api_unreachable`：`CD_LEASE_SECONDS=3`；对 api 进程 `SIGSTOP`；断言内核在 3-5 秒内放弃该 Run。

完成标准

- 破坏补丁 **S1**（删掉心跳 SQL 里的 `AND attempt = $2`）让 `TestHeartbeatStaleAttempt409` 失败，写入 `tests/sabotage/patches/s1-heartbeat-no-attempt.patch`，`make sabotage` 通过。

本步不做

- 回收过期租约（C2）。此时模拟接管后 Run 停在 running，由 C2 解决。

坑

`main_task.cancel()` 只会在下一个 `await` 点生效。LLM 调用是 `await`，所以没问题；但不要在图节点里写长时间的同步 CPU 代码。另外，心跳任务本身必须在 `finally` 里取消，否则 Run 结束后它还在续租，Run 看起来永远活着。

C2 Reaper 与崩溃续跑

依赖：C1 · tag: step-C2

目标：reaper 进程把租约过期的 Run 重新排队（或在尝试次数用尽时失败）；新的内核领取后从 checkpoint 续跑，已完成的节点不重复调用模型。

学习

- “回收”与“执行”分离：reaper 不需要知道 Run 在做什么，只看租约。
- 多个 reaper 同时运行也安全：同样用 `SKIP LOCKED`。
- LangGraph 从 checkpoint 恢复时，会重新执行“崩溃时正在执行的那个节点”。

产出：`cakdesk -role=reaper`

```
-- 每 CD_REAPER_INTERVAL_MS（默认 5000）一个事务：
WITH expired AS (
  SELECT id, lease_owner AS old_owner FROM runs
  WHERE status = 'running' AND lease_expires_at < now()
  ORDER BY lease_expires_at
  FOR UPDATE SKIP LOCKED
  LIMIT 100
)
UPDATE runs r SET
  status      = CASE WHEN r.attempt >= r.max_attempts THEN 'failed' ELSE 'queued' END,
  error_code  = CASE WHEN r.attempt >= r.max_attempts THEN 'max_attempts_exceeded' END,
  error_message = CASE WHEN r.attempt >= r.max_attempts
    THEN format('lease expired on attempt %s', r.attempt) END,
  finished_at = CASE WHEN r.attempt >= r.max_attempts THEN now() END,
  lease_owner = NULL, lease_expires_at = NULL, updated_at = now()
FROM expired
WHERE r.id = expired.id
```

```
RETURNING r.id, r.attempt, r.status, expired.old_owner;
-- 同一事务内为每行追加 run.lease_expired (失败的再追加 run.failed); 提交后发布。
```

产出：内核续跑

- `execute_run` 发现已有 checkpoint 时，先发 `run.resumed {attempt, checkpoint_id, messages}`，然后 `ainvoke(None, cfg)`。
- checkpoint 元数据带上 attempt: `cfg["metadata"] = {"cd_attempt": attempt}`，C3 的僵尸测试用它判断“哪个 attempt 写的”。
- 角色启动: `make run-reaper`。

测试

- `TestReaperRequeuesExpired`：把 `lease_expires_at` 改到过去 → 一次 tick 后 `queued`，事件含 `run.lease_expired{next:"queued"}`。
- `TestReaperFailsAtMaxAttempts`：`attempt = max_attempts` → `failed max_attempts_exceeded`。
- `TestReaperIgnoresLiveLease`、`TestReaperConcurrentSafe`（两个 reaper 同时 tick，每个过期 Run 只产生一条 `lease_expired` 事件）。
- `e2e test_kill19_kernel_resumes`：两个内核 A、B；场景共 5 步，第 3 步延迟 6s；等 A 领取且事件里出现第 2 步的 `tool.finished` 后 `kill -9 A`；断言：Run 最终 `succeeded`、`attempt = 2`；事件依次含 `run.lease_expired`、`run.started{attempt:2}`、`run.resumed`；mock 请求日志中第 0、1 步各只出现一次（已 checkpoint 的节点没有重跑）。

完成标准

- 手动演示：起 api、reaper、两个内核，跑长场景，`kill -9` 正在执行的内核，SSE 里看到接管与续跑。

本步不做

- checkpoint 写入的 fencing (C3) ——此时僵尸内核仍可能写入 checkpoint，这是 C3 要演示并修复的问题。
- 未完成工具调用的处理 (D2)。此阶段的工具只有 `submit_result`，重复执行无害。

C3 Checkpoint fencing

依赖：C2 · 迁移：0004_checkpoint_fence.sql · tag: step-C3

目标：内核写 LangGraph checkpoint 时，在**同一个数据库事务**里校验租约；僵尸内核的 checkpoint 写入被拒绝。这是整个项目最有辨识度的一点。

学习

- 问题本身：Go 侧所有写入都有 fencing，但 checkpoint 是 Python 直接写数据库的。一个被 SIGSTOP 的旧内核醒来后，会把它那份过时的状态写成“最新 checkpoint”，新 attempt 的进度就被污染了。
- PostgreSQL 触发器、`SECURITY DEFINER`、`current_setting(name, true)`、自定义 SQLSTATE。
- 为什么需要 `FOR SHARE`：没有它，“检查通过”和“提交”之间 reaper 可以完成回收、新 attempt 可以读走 checkpoint，旧写入随后提交，形成竞态。有了它，reaper 的 `FOR UPDATE SKIP LOCKED` 会跳过这一行，等写入事务结束后下一轮再回收。
- 为什么用触发器而不是继承 `AsyncPostgresSaver`：`saver` 内部的 pipeline、锁、SQL 随版本变化；触发器不依赖 `saver` 实现，并且 `cd_kernel` 不是表属主，无法禁用它。

产出：0004_checkpoint_fence.sql

```
-- +goose StatementBegin
CREATE FUNCTION public.cd_checkpoint_fence() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public, pg_temp AS $$
DECLARE
    v_run    uuid := nullif(current_setting('cd.run_id', true), '')::uuid;
    v_attempt int := nullif(current_setting('cd.attempt', true), '')::int;
    v_worker text := nullif(current_setting('cd.worker_id', true), '');
BEGIN
    IF v_run IS NULL OR v_attempt IS NULL OR v_worker IS NULL THEN
        RAISE EXCEPTION 'checkpoint write without fence context' USING ERRCODE = 'CD001';
    END IF;
    IF NEW.thread_id <> v_run::text THEN
        RAISE EXCEPTION 'thread % is not fenced run %', NEW.thread_id, v_run USING ERRCODE = 'CD001';
    END IF;
    PERFORM 1 FROM runs
    WHERE id = v_run AND attempt = v_attempt AND lease_owner = v_worker AND status = 'running'
    FOR SHARE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'lease lost: run % attempt % worker %', v_run, v_attempt, v_worker
        USING ERRCODE = 'CD409';
    END IF;
    RETURN NEW;
END $$;
-- +goose StatementEnd
REVOKE ALL ON FUNCTION public.cd_checkpoint_fence() FROM PUBLIC;
```

产出：挂触发器与收紧权限（cd-kernel setup，在 saver.setup() 之后）

```
DO $$ DECLARE t text; BEGIN
    FOREACH t IN ARRAY ARRAY['checkpoints', 'checkpoint_blobs', 'checkpoint_writes'] LOOP
        EXECUTE format('DROP TRIGGER IF EXISTS cd_fence ON lg.%I', t);
        EXECUTE format('CREATE TRIGGER cd_fence BEFORE INSERT OR UPDATE ON lg.%I
            FOR EACH ROW EXECUTE FUNCTION public.cd_checkpoint_fence()', t);
    END LOOP;
END $$;
REVOKE DELETE, TRUNCATE ON ALL TABLES IN SCHEMA lg FROM cd_kernel;
REVOKE INSERT, UPDATE ON lg.checkpoint_migrations FROM cd_kernel;
```

产出：内核设置 fence 上下文

```
# execute_run 里，连接建立后、构建 saver 之前（连接是本 Run 独占的，会话级设置安全）
await conn.execute(
    "SELECT set_config('cd.run_id', %s, false), set_config('cd.attempt', %s, false),"
    "        set_config('cd.worker_id', %s, false)",
    (run_id, str(attempt), worker_id))
# 异常映射：沿 __cause__/_context__ 链找到 psycopg.Error 且 sqlstate == 'CD409' → LeaseLost
```

性质	说明
原子性	触发器与 checkpoint 的 INSERT/UPSERT 在同一事务：要么校验通过且写入，要么都不发生。
不可绕过	cd_kernel 不是 lg 表属主，不能 ALTER TABLE ... DISABLE TRIGGER；改 session_replication_role 需要超级用户。
边界	这是防“过时进程”的 fencing，不是防“恶意内核”的鉴权：内核可以随便设置 cd.*，但仍必须匹配一个 running 的 Run 的当前 attempt 与 owner。
成本	每行一次主键查找加共享行锁。checkpoint 写入频率是每个节点一次，可以忽略。

测试

- `TestCheckpointFenceRejectsStaleAttempt` (Go, 以 `cd_kernel` 连接) : 设置旧 attempt 后 `INSERT INTO lg.checkpoints ...` → `SQLSTATE CD409`。
- `TestCheckpointFenceRequiresContext`、`TestCheckpointFenceThreadMismatch` → `CD001`。
- `TestCheckpointFenceAcceptsCurrentLease`。
- `TestReaperSkipsRunWithCheckpointInFlight` : 连接 K (`cd_kernel`) `BEGIN`、设置上下文、插入一行 checkpoint (持有 `FOR SHARE`) ; 把租约改到过去; reaper tick → 回收 0 行; K `COMMIT` ; reaper tick → 回收 1 行。
- `TestKernelCannotDisableTrigger` : `cd_kernel` 执行 `ALTER TABLE lg.checkpoints DISABLE TRIGGER cd_fence` → `42501`。
- `e2e test_zombie_checkpoint_rejected` : 内核 A 在第 2 步的 6s 延迟中被 `SIGSTOP` ; 等待租约过期、reaper 回收、内核 B 领取 (attempt 2) 并推进; 记录 `metadata->>'cd_attempt' = '1'` 的 checkpoint 行数 N; `SIGCONT` A; 断言 A 日志出现 `lease_lost`, 行数仍为 N, Run 由 B 正常完成且结果正确。

完成标准

- 破坏补丁 S2 (触发器函数直接 `RETURN NEW`) 让 `TestCheckpointFenceRejectsStaleAttempt` 与 `test_zombie_checkpoint_rejected` 失败。
- 破坏补丁 S3 (去掉 `FOR SHARE`) 让 `TestReaperSkipsRunWithCheckpointInFlight` 失败。

坑

goose 默认按分号切分语句, 函数体必须用 `-- +goose StatementBegin/End` 包起来。 `set_config(..., false)` 是会话级, `true` 是事务级——saver 用的是 `autocommit` 连接, 每条语句自成事务, 所以必须用会话级, 也因此每个 Run 必须独占连接, 不能用跨 Run 复用的连接池。

C4 取消与超时

依赖: C3 · 迁移: 0005_run_control.sql · tag: step-C4

目标: 取消与超时都表现为数据库里的状态转换, 由心跳传达、由 reaper 兜底; 不依赖向某个进程发信号。

学习

- 协作式取消: 请求方只写标记, 执行方在安全点检查并自行停止。
- 为什么不加 `cancelling` 状态: 一个时间戳列就能表达“已请求”, 状态机更小, `CHECK` 不变量更简单。

产出: 0005_run_control.sql

```
ALTER TABLE runs ADD COLUMN cancel_requested_at timestampz;
ALTER TABLE runs ADD COLUMN deadline_at timestampz;
UPDATE runs SET deadline_at = created_at + interval '30 minutes' WHERE deadline_at IS NULL;
ALTER TABLE runs ALTER COLUMN deadline_at SET NOT NULL;
CREATE INDEX runs_active_deadline ON runs(deadline_at) WHERE status IN ('queued', 'running');
```

变更	行为
创建 Run	<code>deadline_at = now() + max_duration_s</code> 。

变更	行为
<code>POST /v1/runs/{id}/cancel</code>	事务内 <code>SELECT ... FOR UPDATE</code> : <code>queued</code> → 直接 <code>cancelled</code> (<code>finished_at</code> 、事件 <code>run.cancelled</code>) , 200; <code>running</code> → 若 <code>cancel_requested_at</code> 为空则置 <code>now()</code> , 事件 <code>run.cancel_requested</code> , 202; 终态 → 409 <code>run_finished</code> 。
心跳响应	增加 <code>cancel_requested: bool</code> (<code>RETURNING cancel_requested_at IS NOT NULL</code>) 。
complete	允许 <code>status=cancelled</code> , 但仅当 <code>cancel_requested_at IS NOT NULL</code> , 否则 409 <code>invalid_transition</code> 。
内核	心跳看到取消 → 设 <code>ctx.cancel_requested</code> 。检查点: <code>agent</code> 节点开始、 <code>tools</code> 节点每次调用前。命中则抛 <code>RunCancelled</code> → <code>complete(cancelled)</code> 。D4 起还会中止正在进行的工具请求。
reaper 规则 2	租约过期且 <code>cancel_requested_at</code> 非空 → 直接 <code>cancelled</code> (而不是重新排队) 。
reaper 规则 3	<pre>UPDATE runs SET status='failed', error_code='timeout', error_message='deadline exceeded', finished_at=now(), lease_owner=NULL, lease_expires_at=NULL, updated_at=now() WHERE status IN ('queued','running') AND deadline_at < now() RETURNING id;</pre> 正在运行的内核下一次带租约写入即得到 409, 自然停止。

取消延迟上界: 心跳间隔 (10s) + 当前 LLM 调用时长; D4 之后工具执行会被立即中止。

测试

- `TestCancelQueued`、`TestCancelRunningSetsFlag`、`TestCancelFinished409` 。
- `TestHeartbeatReturnsCancel`、`TestCompleteCancelledRequiresRequest` 。
- `TestReaperExpiredWithCancelBecomesCancelled`、`TestReaperTimeoutFailsQueuedAndRunning` 。
- e2e `test_cancel_running_stops_at_boundary` : 第 2 步延迟 5s, 期间取消; 断言 15s 内 `cancelled`, `cancelled` 之后 mock 无新请求。
- e2e `test_deadline_fails_run` : `max_duration_s=10` , 场景每步延迟 4s → `failed timeout` , 内核随后回到领取状态。

完成标准

- Run 状态机的全部转换 (第 13 章图 13-1) 都有至少一个测试覆盖。

本步不做

- 暂停/恢复 Run、人工重试失败的 Run (写进 `later.md`) 。

10 阶段 D：工具网关与沙箱

STAGE D · D1-D4

每次工具调用都有记录、有归属、有明确的恢复语义

结束时：文件工具经 `os.Root` 限定在工作区；bash 在加固容器里执行并能被超时/取消杀掉整组进程；内核在 bash 执行中被 `kill` 9，恢复后 bash 被报告为 interrupted，同一批次里已完成的 `read_file` 被重放而不是重新执行。

D1 网关持久化与文件工具

依赖：C4 · 迁移：0006_tool_executions.sql · tag: step-D1

目标：每次工具调用在 `tool_executions` 里有一行，主键 `(run_id, tool_call_id)`；已完成的调用再次到达时直接返回存储的结果；上线三个文件工具和工作区文件上传下载。

学习

- 幂等键：由调用方（模型）生成、在 checkpoint 中持久化的 `tool_call_id`，恢复后重发的是同一个 id。
- Go 1.24+ 的 `os.Root`：在一个目录内打开文件，`..`、绝对路径和指向外部的符号链接都会被拒绝（比 `filepath.Clean` + 前缀判断可靠，后者挡不住符号链接）。
- 原子写：临时文件 + `rename`。

产出：0006_tool_executions.sql

```
CREATE TABLE tool_executions (
  run_id      uuid NOT NULL REFERENCES runs(id) ON DELETE CASCADE,
  tool_call_id text NOT NULL CHECK (length(tool_call_id) BETWEEN 1 AND 200),
  tool_name   text NOT NULL,
  args       jsonb NOT NULL,
  args_hash  text NOT NULL,          -- sha256(规范化 JSON)
  idempotent  boolean NOT NULL,      -- 取自工具规格，落库便于审计
  status     text NOT NULL CHECK (status IN ('started', 'succeeded', 'failed', 'interrupted')),
  attempt    int  NOT NULL,          -- 最近一次执行所属 attempt
  executor_id text NOT NULL,         -- 最近一次执行的 api 进程
  exec_count int  NOT NULL DEFAULT 1, -- 真实执行次数（重放不加）
  output     text,
  error_code text,
  started_at timestampz NOT NULL DEFAULT now(),
  finished_at timestampz,
  PRIMARY KEY (run_id, tool_call_id)
);
```

产出：网关流程 (D1 版, D2 补全 started 分支)

```
Tx1 (短事务)
  CheckFence(FOR SHARE)          → 失败 409 lease_lost
  SELECT * FROM tool_executions WHERE (run_id, tool_call_id) FOR UPDATE
  └─ 不存在：校验工具在 config.tools 且已注册；JSON Schema 校验参数
    └─ INSERT status='started', attempt, executor_id; 追加 tool.started; COMMIT
    └─ → 执行 (Tx1 之外)
  └─ args_hash 不同：409 tool_call_conflict (同一个 id 不能换参数)
  └─ succeeded / failed: COMMIT; 追加 tool.finished(replayed:true); 返回存储的结果
执行
  out, err := tool.Execute(ctx, Call{RunID, SessionID, Workspace, Args})
Tx2
  UPDATE tool_executions SET status=$s, output=$o, error_code=$e, finished_at=now()
```

```
WHERE run_id=$1 AND tool_call_id=$2 AND attempt=$a AND executor_id=$me AND status='started'
0 行 → 本次执行已被新 attempt 接管 (D2)，返回 409 lease_lost
追加 tool.finished (系统事件)；COMMIT
响应 {status, output, error_code?, replayed}
```

工具	参数	行为	幂等
list_files	path="."、depth=2 (1-5)	root.FS() + fs.WalkDir；每行 d path/ 或 f path size；最多 500 条，超出注明。	是
read_file	path、offset=0 (行)、limit=400 (1-2000)	首行 [lines a-b of N]；最多 64 KiB；含 NUL 字节 → binary_file。	是
write_file	path、content (≤1 MiB)	root.MkdirAll 父目录；写 .cd-tmp-⟨随机⟩ 后 root.Rename；返回 wrote N bytes to path。覆盖写，同参数重复执行结果相同。	是
submit_result	summary、data?	同 B3。	是

- 所有工具输出由网关统一截断到 16 KiB，附 [truncated N bytes; use read_file with offset/limit]。大内容留在工作区文件里，不进上下文——这是 V1 的“卸载”。
- 路径错误统一映射：越界 → path_outside_workspace；不存在 → not_found。工具错误以 status=failed 返回给模型，不会让 Run 失败。
- 公共接口：PUT /v1/sessions/{id}/files/{path...} (原始 body, ≤10 MiB)、GET /v1/sessions/{id}/files/{path...}、GET /v1/sessions/{id}/files?path=&depth=，同样经 os.Root。
- 查询接口：GET /v1/runs/{id}/tool-calls 返回该 Run 的全部 tool_executions (演示和面试用)。

测试

- TestToolCallReplay：同一 id 调两次，第二次 replayed=true、exec_count=1、输出相同。
- TestToolCallArgsConflict。
- TestPathEscape：../x、/etc/passwd、工作区内指向 /etc 的符号链接，全部 path_outside_workspace。
- TestWriteFileAtomicAndIdempotent、TestReadFileWindowing、TestBinaryFileRejected、TestOutputTruncated。
- TestToolNotInAgentConfig → failed tool_not_in_agent。
- TestSessionFileUploadDownload。

完成标准

- 破坏补丁 S4 (跳过“查已有记录”直接执行) 让 TestToolCallReplay 失败。
- e2e: 上传 3 个文件，场景 list → read → write → submit，Run 成功且工作区出现写入的文件。

本步不做

- started 记录遇到“执行者已死”的处理 (D2)；bash (D3/D4)；Skill 权限 (E1)。

D2 执行语义：先落 started，再按规则恢复

依赖：D1 · tag: step-D2

目标：把“崩溃时正在执行的工具调用”处理得诚实而确定：执行前 started 已提交；恢复时根据执行者是否存活、工具是否幂等，决定重跑、等待或报告 interrupted。

学习

- 为什么做不到 exactly-once：副作用发生在数据库之外（文件、进程），“执行”与“记录完成”之间崩溃，就无法知道副作用是否发生。
- 可以做到的是：**已知完成的不重复（重放）、幂等的放心重跑、不确定的如实告知。**
- 执行者身份（executor_id）：api 每次启动生成新的 id，用它区分“同一进程还在执行”与“执行它的进程已经死了”。

产出：恢复规则（Tx1 中记录已存在时）

已有记录	条件	处理
succeeded / failed	—	重放存储结果（D1）。
started	executor_id == 本进程 且 在飞表中存在	409 tool_call_in_progress。内核每秒重试，直到结果出现。
	其他情况，工具幂等	UPDATE ... SET attempt=\$a, executor_id=\$me, exec_count=exec_count+1, status='started', 提交后重新执行。
	其他情况，工具非幂等	UPDATE ... SET status='interrupted', error_code='executor_lost', output=\$msg, finished_at=now(), 返回 interrupted，不执行。
interrupted	工具幂等	同上，重新执行。
	工具非幂等	返回存储的 interrupted 结果，不执行。

返回给模型的 interrupted 内容（ToolMessage, status=error）：

```

INTERRUPTED: `bash` (call call_2_2_x7k2qa) started in attempt 1 but the process running it
stopped before it finished. Its side effects are unknown. Inspect the workspace (list_files /
read_file) before deciding whether to run it again.

```

产出：在飞表与请求取消

- gateway.inflight：map[key]context.CancelFunc + 互斥锁，执行开始登记、结束删除。
- 执行的 ctx 派生自 HTTP 请求 ctx：调用方（内核）断开 → ctx 取消 → 工具应中止 → Tx2 写 status='interrupted', error_code='cancelled'。原则：**工具执行的生命周期不长于发起它的那次 attempt。**
- 内核侧重试：连接错误/5xx 按 0.5、1、2、4、8s 退避重试同一请求（总计 ≤60s）；tool_call_in_progress 每 1s 重试；lease_lost 抛 LeaseLost。

测试（Go，测试里注册测试替身工具：counterTool{idempotent bool} 往文件追加一行，blockingTool 阻塞直到 ctx 取消或收到信号）

- TestStartedCommittedBeforeExecute：counterTool.Execute 内用另一个连接查询，能看到 started 行。
- TestExecutorLostIdempotentReexecutes：用 executor_id=A 的网关实例插入 started 后“丢弃”它；executor_id=B 的新实例处理同一调用 → 执行，exec_count=2。（两个实例共享数据库，就是 api 重启的真实模型。）
- TestExecutorLostNonIdempotentInterrupted：同上但非幂等 → interrupted，计数文件没有新增行。
- TestInterruptedNonIdempotentStaysInterrupted、TestInterruptedIdempotentReexecutes。
- TestInFlightReturns409：blockingTool 执行中，同 id 第二个请求 → 409 tool_call_in_progress。
- TestRequestCancelMarksInterrupted：取消请求 ctx → 行变为 interrupted/ cancelled。
- TestStaleExecutionCannotWriteResult：A 实例 blockingTool 执行中；模拟新 attempt 由 B 实例把记录改为 interrupted；放行 A → Tx2 0 行 → 409，记录保持 interrupted。

完成标准

- 破坏补丁 S5 (把 started 的 INSERT 挪进 Tx2, 与结果一起提交) 让 `TestStartedCommittedBeforeExecute` 与 `TestExecutorLostNonIdempotentInterrupted` 失败。
- README 用上表写清执行语义, 并明确 “不宣称 exactly-once”。

本步不做

- 真实的非幂等工具 (D4 的 bash)。测试替身只存在于 `_test.go`, 生产注册表里没有它们。

坑

Tx1 必须在执行前**提交**。如果为了省事把 started 和结果放在一个事务里, 崩溃后数据库里什么都没有, 恢复时会把非幂等工具当成全新调用再执行一遍——这正是 S5 模拟的错误。

D3 sandboxd: 会话容器与进程组

依赖: D2 · tag: step-D3

目标: sandboxd 为每个 Session 维护一个加固容器, 提供 exec: 能拿到退出码与截断后的输出, 超时或调用方断开时杀掉整个进程组。

学习

- 容器隔离参数的含义: `--network none`、`--cap-drop ALL`、`no-new-privileges`、`--read-only` + `tmpfs`、`--pids-limit`、`--memory`、`--cpus`、非 root 用户。
- 进程组与会话: `setsid` 让命令成为新会话首进程, `kill -KILL -- -PGID` 一次杀掉它和所有后台子进程。
- `docker exec` 的陷阱: 客户端断开**不会**杀掉容器里的进程。
- 为什么只让一个进程持有 `docker.sock`: 拿到 `docker.sock` 约等于拿到宿主机 root。

产出: 镜像 `sandbox/Dockerfile`

```
FROM python:3.12-slim-bookworm
RUN apt-get update && apt-get install -y --no-install-recommends bash procps util-linux \
  && rm -rf /var/lib/apt/lists/*
WORKDIR /workspace
CMD ["sleep", "infinity"]
# make sandbox-image → cakdesk-sandbox:dev
```

产出: sandboxd 接口 (HTTP over unix socket `var/run/sandboxd.sock`, 权限 0600)

接口	行为
<code>POST /v1/sandboxes/ensure {session_id, workspace_dir, skills_dir, skills_hash}</code>	按名字 <code>cd-sess-<session_id></code> 查容器: 运行中且标签 <code>cakdesk.skills_hash</code> 相同 → 复用; 否则强制删除后重建。同一 Session 的并发 <code>ensure</code> 用 <code>singleflight</code> 合并。返回 <code>{container_id, created}</code> 。
<code>POST /v1/sandboxes/{session_id}/exec {exec_id, command, timeout_s, max_output_bytes}</code>	见下方算法。返回 <code>{exit_code, stdout, stderr, timed_out, truncated, duration_ms}</code> 。
<code>DELETE /v1/sandboxes/{session_id}</code> 、 <code>GET /v1/sandboxes</code>	删除; 列出 (调试)。

容器创建参数

参数	值
Image / Cmd	cakdesk-sandbox:dev / sleep infinity
User	CD_SANDBOX_UID:CD_SANDBOX_GID, 默认取 sandboxd 进程的 uid/gid (这样容器写出的文件属于宿主机用户, api 的 os.Root 可以读写)。不得为 0。
NetworkMode	none
CapDrop / SecurityOpt	ALL / no-new-privileges:true
ReadOnlyRootfs / Tmpfs	true / /tmp: rw,nosuid,nodev,size=64m
Resources	Memory 512 MiB (MemorySwap 相同, 禁 swap)、NanoCPUs 1e9、PidsLimit 256
Init	true (tini 回收僵尸进程)
Mounts	workspace_dir → /workspace (rw); skills_dir → /skills (ReadOnly)
Env / Labels	HOME=/tmp / cakdesk.managed=true、cakdesk.session、cakdesk.skills_hash

exec 算法

```

pidfile := "/tmp/cd-exec-" + execID + ".pid"
cmd := ["setsid", "-w", "bash", "-c", `echo $$ > "$CD_PIDFILE"; exec bash -c "$CD_CMD"`]
env := ["CD_PIDFILE=" + pidfile, "CD_CMD=" + command]
ExecCreate(AttachStdout, AttachStderr, WorkingDir=/workspace) → ExecAttach
go stdcopy.StdCopy(limitedBuf(stdout), limitedBuf(stderr), attach.Reader) // 每路上限, 超出计数
select {
    case <-copyDone:           // 正常结束
    case <-time.After(timeout): timedOut = true; killGroup()
    case <-ctx.Done():         killGroup()           // 调用方断开 / 取消
}
killGroup: ExecCreate(["sh", "-c", `kill -KILL -- -$(cat "$CD_PIDFILE")" 2>/dev/null`]) 并等待; 随后最多再等 5s 让 attach 结束
ExecInspect → ExitCode; 最后 exec rm -f pidfile

```

- 生命周期: 进程内 lastUsed 表, 每 60s 删除空闲超过 CD_SANDBOX_IDLE_SECONDS (600) 的容器。启动时对账: 删除所有 cakdesk.managed=true 的容器。容器只是缓存, 状态在宿主机工作区。
- skills.TreeHash(dir): 对排好序的(相对路径、文件模式、内容 sha256)序列再做 sha256; D3 用于容器标签, E1 用于 Skill 快照。
- api 侧 internal/sandboxclient: http.Client 的 DialContext 指向 unix socket。
- archtest 规则生效: 仅 internal/sandboxd 可 import Docker SDK。

测试 (//go:build docker, 在 make e2e 中运行)

- TestEnsureHardenedContainer: inspect 断言上表每一项。
- TestEnsureReuse、TestEnsureRecreateOnSkillsHashChange、TestReconcileRemovesManaged、TestIdleCleanup (空闲阈值 1s)。
- TestExecNoNetwork: python3 -c "import socket; socket.create_connection(('1.1.1.1', 80), 3)" 退出码非 0。
- TestExecReadOnlyRoot: touch /etc/x 失败; touch /workspace/x 成功且宿主机文件属主为当前用户; touch /skills/x 失败。
- TestExecTimeoutKillsGroup: sleep 300 & sleep 300, timeout 2 → timed_out=true; 随后 pgrep -c sleep 为 0。
- TestExecCancelKillsGroup: 取消 ctx 后同样无残留进程。
- TestExecForkBombContained: fork 炸弹 + timeout 5 → 结束后容器仍能执行 echo ok。
- TestExecOutputTruncated、TestExecExitCode。

完成标准

- `make run-sandboxd` 后，用 `curl --unix-socket` 演示：无网络、只读根、后台子进程被超时杀掉。

坑

`setsid` 若发现自己已经是进程组首进程，会 `fork` 后**立刻退出**，`docker exec` 随即结束而命令还在后台跑——必须加 `-w`（等待子进程并返回其退出码）。命令里自己再 `setsid` 的进程会逃出进程组，这是已知限制，写进 README；容器被删除时它们仍会被清理。

D4 bash 工具与端到端崩溃恢复

依赖：D3 · tag: step-D4

目标：bash 作为第一个真实的非幂等工具接入网关；取消与断开能从内核一路传到容器里的进程组；完成项目核心演示的自动化测试。

学习

- ctx 取消链：内核 HTTP 断开 → api 请求 ctx → sandboxclient 请求 → sandboxd 处理器 ctx → killGroup。每一跳都不需要额外代码，只要 ctx 一路传下去。
- 超时分层：内层必须先于外层超时，否则外层先断开，内层结果无处可写。

产出

项	细节
<code>tools/bash.go</code>	参数 <code>{command: string ≤10000, timeout_s: 1-600 默认 60}</code> ， <code>Idempotent=false</code> 。执行： <code>Ensure(session) → Exec(ctx, exec_id = run_id:tool_call_id:exec_count)</code> 。输出格式： <code>exit_code: N</code> （超时加一行 <code>timed out after Ns</code> ） <code>--- stdout --- ... --- stderr --- ...</code> 非 0 退出码仍是 <code>status=succeeded</code> （工具运行了，结果交给模型判断）； <code>sandboxd</code> 不可用 → <code>failed sandbox_unavailable</code> 。
超时分层	容器内 <code>timeout_s < api 调 sandboxd 的 HTTP 超时 timeout_s + 15 < 内核调网关的 HTTP 超时 timeout_s + 30</code> 。
内核取消	心跳发现取消（C4）时，除了在边界检查外，还会取消当前工具请求所在的 <code>asyncio</code> 任务 → 连接关闭 → 网关按 D2 标记 <code>interrupted/ cancelled</code> → 内核 <code>complete(cancelled)</code> 。

测试 (e2e)

- `test_kill9_kernel_during_bash`（核心演示的自动化版本）：场景 `csv-report`，第 3 步是一条消息内的 `[read_file a, read_file b, bash(... && sleep 15)]`。等待 `tool.started{name:"bash"}` 后 `kill -9` 内核 A，内核 B 接手。断言：
 - bash 的 `tool_executions`：`status=interrupted`、`exec_count=1`；容器内 `pgrep sleep` 为空（A 断开时已被杀）。
 - 两个 `read_file`：`exec_count=1`，`attempt 2` 的事件里它们的 `tool.finished.replayed=true`。
 - 模型下一步（场景第 4 步 `test -s out/summary.csv || python3 ...aggregate.py ...`）执行成功；`Run succeeded`，`out/summary.csv` 内容正确。
- `test_kill9_api_during_bash`：bash 执行中 `kill -9 api` 并重启（新 `executor_id`）；内核重试同一调用 → `interrupted/ executor_lost`；容器内无残留进程（`sandboxd` 检测到 api 连接断开）。
- `test_cancel_during_bash`：bash `sleep 60` 中取消 → 15s 内 `cancelled`，容器无残留进程，记录 `interrupted/ cancelled`。

- `test_bash_timeout` : `timeout_s=2` 的 `sleep 30` → succeeded、输出含 `timed out` , Run 继续。

完成标准

- 这四个 e2e 连续跑 5 次全部通过（排除时序偶发）。
- 第 17 章演示的第 1-4 幕此时已能手动完成。

本步不做

- bash 权限约束 (E1) : 此步 bash 只要在 `config.tools` 中即可调用。

11 阶段 E: Skill 权限与最小上下文

STAGE E · E1-E3

模型能做什么由执行点决定；长任务不被上下文和故障拖垮

结束时：未加载相应 Skill 时 bash 被网关拒绝；LLM 限流自动重试；预算与循环保护的计数跨崩溃保留；上下文超过阈值时压缩而不破坏工具调用配对。

E1 Skill 与权限硬约束

依赖：D4 · 迁移：0007_run_skills.sql · tag: step-E1

目标：Skill 以 SKILL.md 定义，声明 allowed-tools；模型通过 load_skill 激活；网关在每次执行前计算有效工具集合并拒绝越权调用。

学习

- “提示词里的约束只是建议，执行点的检查才是约束”。
- Anthropic Agent Skills 的 SKILL.md 约定（YAML frontmatter + Markdown 正文、渐进式加载）。
- 配置快照：Run 创建时锁定 Skill 的哈希，运行中 Skill 被修改能被检测到。

产出：Skill 格式

```
skills/csv-report/SKILL.md
---
name: csv-report
description: Aggregate the CSV files under data/ into out/summary.csv and write out/report.md.
allowed-tools: [bash, write_file]
---
# CSV report
1. `list_files data` to see the inputs; `read_file` a few of them to check the columns.
2. Run `python3 /skills/csv-report/scripts/aggregate.py data out/summary.csv`.
3. Write a short markdown report to `out/report.md` with `write_file`.
4. Call `submit_result` with the totals.
```

规则	细节
解析 (internal/skills)	frontmatter 位于首尾两行 --- 之间；name 必须等于目录名且匹配 <code>^[a-z0-9][a-z0-9-]{1,62}\$</code> ；description 1-1024 字符；allowed-tools $\subseteq$ 工具注册表；正文 ≤ 32 KiB。
Agent 配置校验 (新增)	skills 中每个 Skill 必须存在且合法；其 allowed-tools $\subseteq$ Agent 的 tools（否则配置自相矛盾，400 invalid_config）；若 tools 含 load_skill 以外的非基线工具，但没有任何 Skill 允许它，也拒绝（永远用不上）。
Run 快照	创建 Run 时写入 <code>config.resolved_skills = [{name, hash, description, allowed_tools}]</code> ，hash 用 D3 的 TreeHash。
系统提示	内核在 system_prompt 后追加：可用 Skill 列表（name + description）以及“基线外的工具需要先 load_skill”。

产出：0007_run_skills.sql 与 load_skill

```
CREATE TABLE run_skills (
  run_id      uuid NOT NULL REFERENCES runs(id) ON DELETE CASCADE,
  skill_name  text NOT NULL,
```

```
skill_hash    text NOT NULL,
tool_call_id text NOT NULL,
loaded_at     timestampz NOT NULL DEFAULT now(),
PRIMARY KEY (run_id, skill_name)
);
```

- `load_skill {name}`，幂等。`name` 必须在 `resolved_skills` 中（否则 `failed skill_not_in_agent`）；重新计算磁盘上的 `TreeHash`，与快照不同 → `failed skill_changed`；否则 `INSERT ... ON CONFLICT DO NOTHING`，追加 `skill.loaded`，返回 `SKILL.md` 正文（去掉 `frontmatter`）。
- 容器挂载的是整个 `skills/` 目录（只读），`skills_hash` 变化时 D3 会重建容器。

产出：权限检查（在 Tx1 中，新执行与重新执行前都做；重放不做）

```
BASELINE = {load_skill, list_files, read_file, submit_result}
effective(run) = config.tools ∩ (BASELINE ∪ { s.allowed_tools : s ∈ run_skills(run) })
```

不在 `effective` 中：

```
INSERT tool_executions(status='failed', error_code='tool_not_allowed', exec_count=0, output=$msg)
(落库使重放结果确定)；追加 tool.finished(status:"failed")；返回 failed
$msg = "tool `bash` is not allowed yet. Skills that allow it: csv-report. Call load_skill first."
```

模型能看到 Agent 配置的全部工具定义（不按有效集合过滤），这样“越权尝试”会真实发生并被网关拦截，演示与测试都更直接。

测试

- `TestEffectiveTools`：表驱动（无 Skill、单 Skill、多 Skill、Skill 允许但 Agent 未配置）。
- `TestBashDeniedWithoutSkill`、`TestBashAllowedAfterLoadSkill`、`TestDeniedCallReplaysDenied`（同 id 再来一次仍是拒绝，而不是因为此时已加载 Skill 而执行）。
- `TestLoadSkillNotInAgent`、`TestLoadSkillHashChanged`、`TestSkillParse`（每条规则一个反例）、`TestAgentConfigSkillToolsSubset`。
- `e2e test_permission_scenario`：场景先调 `bash` → 被拒；再 `load_skill` → `bash` 成功；事件依次可见。

完成标准

- 破坏补丁 **S6**（有效集合直接返回 `config.tools`）让 `TestBashDeniedWithoutSkill` 失败。
- `csv-report` Skill 与 `aggregate.py` 可用，D4 的 `e2e` 更新为先 `load_skill`。

本步不做

- Skill 上传/版本管理接口、Skill 依赖、Skill 卸载、LLM 审查 Skill。

E2 LLM 重试、预算与循环保护

依赖：E1 · tag: step-E2

目标：三个中间件：可重试错误自动退避重试；LLM 调用次数、工具调用次数与 token 总量有上限；模型原地打转时先警告后失败。计数存在图状态里，崩溃后不清零。

学习

- 指数退避 + 抖动；Retry-After。
- “计数器放在 checkpoint 里”：进程内变量会随 `kill -9` 丢失，预算就能被崩溃绕过。

产出：状态扩展

```
class RunState(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]
    result: dict | None
    nudges: int
    llm_calls: int          # E2
    tool_calls: int         # E2
    total_tokens: int       # E2 (来自 usage.total_tokens)
    budget_warned: bool     # E2, 80% 时告警一次
    loop_sig: str | None    # E2, 上一条 AIMessage 的工具调用签名
    loop_streak: int        # E2
```

```
async def agent_node(state, config):
    ctx = run_ctx(config)
    ctx.check_cancel()
    budget.check_before_model(state, ctx.limits)          # 超限 → RunFailed("budget_exceeded")
    msgs, compact_update = await compaction.maybe(state, ctx) # E3
    extra = loop_guard.warning_if_needed(state)            # 返回 [] 或 [HumanMessage("[runtime] ...")]
    ai = await retry.invoke(ctx, msgs + extra)
    return {"messages": extra + [ai], "llm_calls": state["llm_calls"] + 1,
            "total_tokens": state["total_tokens"] + usage(ai), **compact_update}
```

中间件	规则
retry	可重试：RateLimitError、APIStatusError 且 status ≥ 500、APITimeoutError、APIConnectionError。第 n 次（从 0 开始）等待 $\min(16, 2^n) \times U(0.8, 1.2)$ 秒；有 retry-after 则取它（上限 30s）；最多重试 5 次；每次发 llm.retry。用尽 → llm_unavailable；其他 4xx → 立即 llm_bad_request。测试中 <code>CD_RETRY_BASE_MS=10</code> 缩放。
budget	模型调用前：llm_calls ≥ max_llm_calls 或 total_tokens ≥ max_total_tokens → failed budget_exceeded（message 注明哪一项）。tools 节点每次调用前：tool_calls ≥ max_tool_calls 同理。任一项首次达到 80% → guard.warning(kind:"budget")。
loop_guard	签名 = 最后一条 AIMessage 的工具调用按 (name, json.dumps(args, sort_keys=True)) 排序后的 sha256。与 loop_sig 相同则 streak+1，否则重置为 1（在路由到 tools 前更新）。streak = 3 → 下次模型调用前插入 [runtime] You have issued the same tool calls 3 times in a row with no progress. Change approach or call submit_result。并发 guard.warning(kind:"loop")；streak ≥ 5 → failed loop_detected。

已知误差：LLM 调用返回后、checkpoint 写入前崩溃，这一次调用不计入预算。每次崩溃最多少计一次，且被 max_attempts 限制，可以接受并写进 README。

测试 (pytest, 真实 mock)

- test_retry_429_then_success：errors times:2 → 成功；2 条 llm.retry；mock 日志中该步 3 次请求。
- test_retry_exhausted_fails、test_400_fails_fast、test_retry_after_respected。
- test_budget_llm_calls：max_llm_calls=3 → 恰好 3 次模型请求后 failed。
- test_budget_survives_crash：max_llm_calls=4，第 2 次调用后 kill -9 内核 → 全程模型请求总数 ≤ 5（4 + 最多 1 次少计），而不是 6 以上。
- test_loop_warn_then_fail（repeat: 10）、test_loop_recovers_after_warning（repeat: 3 后换工具 → 成功且只有 1 条警告）。

完成标准

- 三个中间件各自有单元测试（不起进程）和至少一个 e2e。

本步不做

- 模型降级/切换、费用计算、按 Session 的预算。

E3 上下文压缩

依赖：E2 · tag: step-E3

目标：消息总量超过阈值时，把中段历史压缩为一条摘要，保留开头的系统/任务消息与最近的若干条消息，且绝不拆开 AIMessage 与它的 ToolMessage。

学习

- 工具调用配对约束：OpenAI 兼容接口要求每个带 `tool_calls` 的 assistant 消息后面紧跟全部对应的 tool 消息，拆开会直接 400。
- LangGraph 的 `RemoveMessage` 与 `REMOVE_ALL_MESSAGES`：用 reducer 重写整个消息列表。

产出： `middleware/compaction.py`

```
estimate(msgs) = Σ (len(content) + len(json.dumps(tool_calls))) // 4    # 与 mock 的计数一致；近似值

maybe(state, ctx):
    if estimate(messages) <= compact_threshold_tokens: return messages, {}
    head = messages[:2]                # system + 第一条任务消息
    i = len(messages) - keep_last_messages
    while i > 2 and isinstance(messages[i], ToolMessage): i -= 1    # 向前扩到对应的 AIMessage
    tail, middle = messages[i:], messages[2:i]
    if not middle: return messages, {}
    summary = await retry.invoke(ctx, [
        SystemMessage("[cd:summarize]] Summarize the work so far for your future self: files "
            "inspected, commands run and their results, decisions made, what remains. "
            "At most 300 words."),
        HumanMessage(render_as_text(middle))]
    new = head + [HumanMessage("[runtime summary of earlier steps]\n" + summary.content)] + tail
    emit context.compacted{removed: len(middle), kept: len(tail), summary_chars}
    return new, {"messages": [RemoveMessage(id=REMOVE_ALL_MESSAGES), *new],
        "llm_calls": +1 (摘要调用计入预算)}
```

- 压缩发生在 agent 节点内部，与本次模型回复一起被 checkpoint；压缩中途崩溃，恢复后整个节点重做，结果一致。
- 旧摘要如果落在中段会被再次摘要，不需要特殊处理。
- mock 的步骤推断依赖最后一对 AI/Tool 消息 (A3)，压缩保留 tail，所以场景能继续推进——`test_step_from_history` 已覆盖。

测试

- `test_cut_never_splits_tool_pairs`：随机生成 500 个合法消息序列与参数组合，对每个输出断言：每条 ToolMessage 之前都有其 AIMessage，每个 AIMessage 的所有 tool_call_id 都有回应。
- `test_compaction_triggers_e2e`：`compact_threshold_tokens=2000`，读 20 个文件的长场景 → 至少一条 `context.compacted`，Run 成功，最后一次模型请求的消息数 $\leq 2 + 1 + \text{keep_last} + \text{当批工具消息数}$ 。
- `test_compaction_resume`：压缩后 `kill -9`，续跑时读取到的是压缩后的消息列表。

完成标准

- 演示场景（20 个 CSV）在默认阈值下触发一次压缩并成功完成。

本步不做

- 精确 tokenizer、跨会话记忆、工具输出卸载到文件（网关的 16 KiB 截断 + 工作区文件已足够）。

12 阶段 F：破坏测试、演示与文档

STAGE F · F1

把“可靠”变成可以被别人复现的证据

不加新功能。只做收口：破坏测试全集、一键演示、README 与面试文档。

F1 收口

依赖：E3 · tag: step-F1 (= v1.0)

产出

文件	内容
tests/sabotage/run.sh	对每个补丁： <code>git worktree add /tmp/cd-sab-XX HEAD</code> → <code>git apply</code> → 运行该补丁对应的测试命令 → 期望失败 → 删除 <code>worktree</code> 。最后输出表格（补丁、期望失败的测试、结果）。任何补丁没有让测试失败，脚本退出 1。 <code>make sabotage</code> 。
tests/sabotage/patches/	S1-S8（第 16 章）。补丁要尽量小，一眼能看懂“拿掉了哪条保证”。
demo/seed_csv.py	固定随机种子 42，生成 20 个 <code>sales_XX.csv</code> （ <code>date, region, product, units, unit_price</code> ，每个 200 行），通过文件上传接口放进 Session 的 <code>data/</code> 。
skills/csv-report/scripts/aggregate.py	按 region 汇总行数、units、revenue，写 <code>out/summary.csv</code> ；输出确定，重复执行结果相同。
demo/demo.sh	第 17 章的脚本化版本，每一幕之间暂停等待回车。
README.md	定位；架构图；三条主线各一段，每段链接到对应测试与破坏补丁；执行语义表（D2）；信任边界；快速开始；演示；限制与不做的事。
docs/interview.md	第 18 章的五个问题及回答要点、可以当场打开的代码位置。
docs/later.md	V1 期间记下的想法（LISTEN/NOTIFY、控制台、审批等待、多租户...）。

完成标准

- 全新克隆：`make up` && `make sandbox-image` && `make migrate` && `make test` && `make e2e` && `make sabotage` 全部通过。
- `demo.sh` 从零到最后一幕无需手工干预（除按回车）。
- README 中每一条可靠性声明都能指向一个测试名和一个破坏补丁。

13 数据模型汇总（最终 DDL）

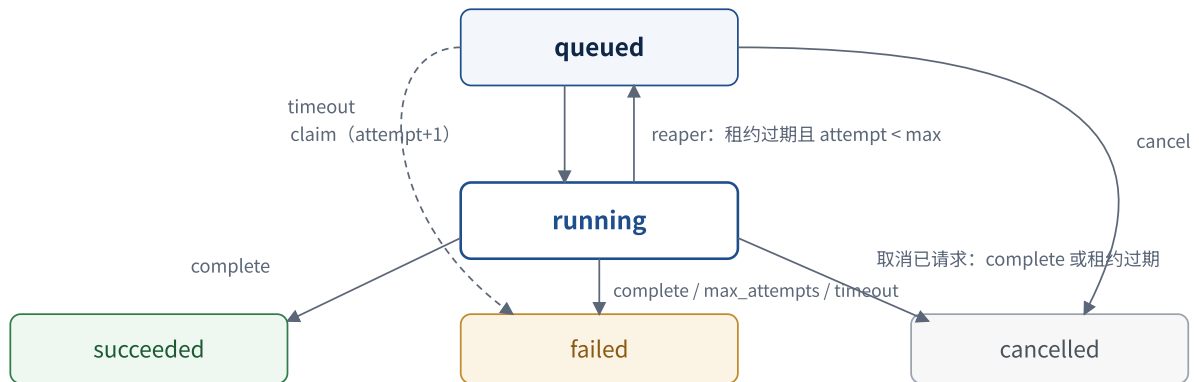


图 13-1 Run 状态机。取消请求不是状态，而是 `cancel_requested_at` 列。

13.1 迁移清单

文件	步骤	内容
0001_agents.sql	A2	schema 权限、默认权限、lg schema、agents、agent_versions
0002_sessions_runs.sql	B1	sessions、runs（含租约字段）、三个索引
0003_run_events.sql	B4	runs.last_seq、run_events
0004_checkpoint_fence.sql	C3	cd_checkpoint_fence()
0005_run_control.sql	C4	cancel_requested_at、deadline_at
0006_tool_executions.sql	D1	tool_executions
0007_run_skills.sql	E1	run_skills
cd-kernel setup	B3 / C3	LangGraph checkpoint 表；fencing 触发器；收紧 cd_kernel 权限

13.2 runs 表最终形态

```

runs(
  id uuid PK, session_id uuid FK, agent_id uuid, agent_version int, FK(agent_id, agent_version),
  config jsonb,           -- 含 resolved_skills
  input text,
  status text CHECK IN (queued, running, succeeded, failed, cancelled),
  attempt int,           -- fencing token
  max_attempts int,
  lease_owner text, lease_expires_at timestampz,
  cancel_requested_at timestampz, deadline_at timestampz NOT NULL,
  last_seq bigint,
  result jsonb, error_code text, error_message text,
  created_at, started_at, finished_at, updated_at
)

```

不变量: `status='running'` ⇔ `lease_owner` 与 `lease_expires_at` 非空
`status` 为终态 ⇔ `finished_at` 非空
 每个 session 至多一个 queued/running 的 run（部分唯一索引）

13.3 权限矩阵

对象	cd_migrate	cd_app	cd_kernel
public 业务表	属主	SELECT/INSERT/UPDATE/DELETE (agent_versions 无 UPDATE/DELETE)	无
lg.checkpoints / blobs / writes	属主	无	SELECT/INSERT/UPDATE (受 触发器约束)
lg.checkpoint_migrations	属主	无	SELECT
<code>cd_checkpoint_fence()</code>	属主 (SECURITY DEFINER 以其身份读 runs)	—	— (由触发器调用)

14 接口汇总

14.1 公共 API (:7310, Authorization: Bearer \$CD_API_KEY)

接口	步骤	说明
GET /healthz	A1	免鉴权; A2 起含 DB ping
POST /v1/agents、GET /v1/agents、GET /v1/agents/{id}	A2	
POST /v1/agents/{id}/versions、GET /v1/agents/{id}/versions/{v}	A2	发布即成为当前版本; 同配置幂等
POST /v1/sessions、GET /v1/sessions/{id}	B1	
POST /v1/sessions/{id}/runs、GET /v1/sessions/{id}/runs	B1	409 session_busy
GET /v1/runs/{id}、GET /v1/runs/{id}/config	B1	
GET /v1/runs/{id}/events、GET /v1/runs/{id}/events/stream	B4	SSE, Last-Event-ID
POST /v1/runs/{id}/cancel	C4	200 / 202 / 409
PUT GET /v1/sessions/{id}/files/{path...}、GET /v1/sessions/{id}/files	D1	经 os.Root
GET /v1/runs/{id}/tool-calls	D1	tool_executions 视图

14.2 内部 API (127.0.0.1:7312, Authorization: Bearer \$CD_INTERNAL_TOKEN)

接口	步骤	请求 → 响应
POST /internal/runs/claim	B2	{worker_id, wait_ms} → 200 {run_id, session_id, attempt, input, config, tools, lease_seconds, heartbeat_seconds} 204
POST /internal/runs/{id}/heartbeat	C1	{worker_id, attempt} → {lease_expires_at, cancel_requested} 409
POST /internal/runs/{id}/events	B4	{worker_id, attempt, events:[{type,payload}]} → {first_seq, last_seq} 409
POST /internal/runs/{id}/tool-calls	B3	{worker_id, attempt, tool_call_id, name, args} → {status, output, error_code?, replayed} 409 lease_lost / tool_call_in_progress / tool_call_conflict
POST /internal/runs/{id}/complete	B2	{worker_id, attempt, status, result?, error?} → 200 409

14.3 sandboxd (unix socket)

POST /v1/sandboxes/ensure、POST /v1/sandboxes/{session_id}/exec、DELETE /v1/sandboxes/{session_id}、GET /v1/sandboxes, 见 D3。

15 故障模型与恢复矩阵

故障	系统行为	证明
内核在 LLM 调用中被 kill -9	租约到期 → reaper 重新排队 → 新内核从上一个 checkpoint 续跑，该次 LLM 调用重做。	<code>test_kill9_kernel_resumes</code>
内核在工具批次中被 kill -9	已完成的调用重放；执行中的调用因连接断开被中止并标记 interrupted；幂等的重跑，非幂等的报告给模型。	<code>test_kill9_kernel_during_bash</code>
内核被暂停后恢复（僵尸）	心跳、事件、工具、完成返回 409；checkpoint 写入被触发器拒绝；僵尸自行放弃。	<code>test_zombie_checkpoint_rejected</code>
内核与 api 之间网络中断	内核重试；超过 <code>lease_seconds</code> 未能心跳则自我隔离；租约由 reaper 回收。	<code>test_self_fence_when_api_unreachable</code>
api 在工具执行中崩溃	内核重试到新的 api；记录属于已死的执行者 → 按 D2 规则处理；sandboxd 因连接断开杀进程组。	<code>test_kill9_api_during_bash</code>
reaper 停止	正常 Run 不受影响；崩溃的 Run 停在 running 直到 reaper 恢复。多个 reaper 可同时运行。	<code>TestReaperConcurrentSafe</code>
sandboxd 崩溃/重启	进行中的 bash 返回 failed <code>sandbox_unavailable</code> ；重启时到账删除全部托管容器，下次 ensure 重建；工作区文件不受影响。	<code>TestReconcileRemovesManaged</code>
Redis 不可用	Run 正常执行完成；SSE 降级为每秒查库。	<code>TestSSEWorksWithoutRedis</code>
PostgreSQL 不可用	一切暂停：心跳失败 → 内核自我隔离；恢复后 reaper 回收，Run 续跑。不追求此时可用。	手动验证，写入 README
LLM 429/5xx	退避重试 5 次，失败则 Run 失败 <code>llm_unavailable</code> 。	<code>test_retry_*</code>
模型死循环 / 超预算	警告后失败 <code>loop_detected</code> / <code>budget_exceeded</code> ；计数跨崩溃保留。	<code>test_loop_*</code> 、 <code>test_budget_survives_crash</code>
模型越权调用工具	网关拒绝 <code>tool_not_allowed</code> ，Run 继续。	<code>TestBashDeniedWithoutSkill</code>
模型执行破坏性命令	只影响本 Session 工作区；无网络、只读根、资源受限。	D3 安全测试组

16 破坏测试清单

每个补丁拿掉一条保证，对应的测试必须失败。它证明的是“测试真的在测这件事”，而不是“代码碰巧通过”。

编号	补丁（拿掉什么）	必须失败的测试	引入	对应保证
S1	心跳 SQL 去掉 AND attempt = \$2	TestHeartbeatStaleAttempt409	C1	旧 attempt 不能续租
S2	fence 触发器直接 RETURN NEW	TestCheckpointFenceRejectsStaleAttempt 、 test_zombie_checkpoint_rejected	C3	僵尸不能写 checkpoint
S3	fence 触发器去掉 FOR SHARE	TestReaperSkipsRunWithCheckpointInFlight	C3	检查与提交之间不能被回收
S4	网关跳过已有记录查询	TestToolCallReplay	D1	已完成的调用只执行一次
S5	started 与结果同事务提交	TestStartedCommittedBeforeExecute 、 TestExecutorLostNonIdempotentInterrupted	D2	非幂等工具不被静默重跑
S6	有效工具集合 = config.tools	TestBashDeniedWithoutSkill	E1	Skill 权限硬约束
S7	删除 runs_one_active_per_session 索引	TestOneActiveRunPerSession	B1	一个工作区只有一个写者
S8	预算计数改为进程内变量（不进状态）	test_budget_survives_crash	E2	崩溃不能绕过预算

17 演示脚本

四个终端：A 跑 api + reaper + sandboxd；B 跑内核 1；C 跑内核 2；D 执行命令与 SSE。整个演示约 8 幕，每幕都能在 SSE 与 SQL 中看到证据。

幕	操作	观众看到
0	<code>make up migrate sandbox-image</code> ；启动所有进程； <code>demo/seed_csv.py</code>	Agent <code>analyst v1</code> 、Session、工作区里 20 个 CSV。
1	创建 Run（场景 <code>csv-report</code> ），D 终端 <code>curl -N .../events/stream</code>	<code>run.queued</code> → <code>run.started{attempt:1}</code> → <code>message.assistant (bash)</code> → <code>tool.finished{failed, tool_not_allowed}</code> → <code>load_skill</code> → <code>skill.loaded</code> 。
2	出现 <code>tool.started{name:"bash"}</code> （带 <code>sleep 15</code> ）时， <code>kill -9</code> 内核 1	SSE 停顿；约 30 秒后 <code>run.lease_expired</code> → <code>run.started{attempt:2}</code> （内核 2）→ <code>run.resumed</code> 。
3	继续观察	两个 <code>read_file</code> 的 <code>tool.finished{replayed:true}</code> ； <code>bash</code> 的 <code>tool.finished{status:"interrupted"}</code> ；模型执行 <code>test -s</code> ；写报告；压缩一次； <code>run.succeeded</code> 。
4	<code>GET /v1/runs/{id}/tool-calls</code> 与 SQL	<code>bash: interrupted</code> 、 <code>exec_count 1</code> ； <code>read_file: exec_count 1</code> ； <code>out/summary.csv</code> 正确。
5	新 Run；内核 1 执行时 <code>kill -STOP</code> ；等待内核 2 接手并完成； <code>kill -CONT</code> 内核 1	内核 1 日志： <code>heartbeat 409</code> 、 <code>checkpoint</code> 写入 <code>CD409</code> → <code>lease_lost</code> ，放弃。SQL： <code>attempt 1</code> 的 <code>checkpoint</code> 行数在 <code>CONT</code> 前后相同。
6	Run 中模型执行 <code>curl https://example.com</code> 、 <code>touch /etc/x</code>	<code>bash</code> 输出里是网络不可达与只读文件系统错误；Run 照常继续。
7	运行中 <code>POST /v1/runs/{id}/cancel</code>	<code>run.cancel_requested</code> → <code>bash interrupted/cancelled</code> → <code>run.cancelled</code> ；容器内无残留进程。
8	<code>make sabotage</code>	S1-S8 每一条都让指定测试失败的表格。

```
-- 演示用 SQL
SELECT seq, type, attempt, payload->>'name' AS tool, payload->>'status' AS status,
       payload->>'replayed' AS replayed
FROM run_events WHERE run_id = :run ORDER BY seq;

SELECT tool_call_id, tool_name, status, attempt, exec_count, error_code
FROM tool_executions WHERE run_id = :run ORDER BY started_at;

SELECT metadata->>'cd_attempt' AS attempt, count(*)
FROM lg.checkpoints WHERE thread_id = :run::text GROUP BY 1;
```

18 面试要点

问题	回答要点	当场打开
1. worker 被 kill -9, Run 怎么继续?	租约 + 心跳 + reaper; PostgreSQL SKIP LOCKED 领取; thread_id=run_id 从 checkpoint 续跑; checkpoint 粒度是节点, 所以崩溃节点会重做。	C2 SQL、worker.py
2. 旧 worker 醒来会不会写坏状态?	fencing token = attempt; 所有 Go 侧写入是条件 UPDATE; Python 直写的 checkpoint 用触发器在同一事务校验, FOR SHARE 堵住检查与提交之间的竞态; 为什么选触发器而不是继承 saver。	0004_checkpoint_fence.sql、S2/S3
3. 工具会不会被执行两次?	不宣称 exactly-once; started 先提交; 已完成重放; 幂等重跑; 非幂等 interrupted 交给模型; executor_id 判断执行者存活; 执行生命周期绑定请求。	D2 规则表、S4/S5
4. 怎么防止模型乱来?	权限在执行点: 有效工具 = 配置 $\cap$ (基线 $\cup$ 已加载 Skill); 拒绝也落库保证重放确定; bash 在无网络、只读根、降权、限资源容器里, 进程组超时杀; 信任边界: Skill 可信、模型不可信。	网关 effective()、D3 参数表
5. 为什么这样选型?	PostgreSQL 做队列避免双写一致性问题; Redis 只做加速; Go 做控制面、Python 只做 Agent 语义; 只有 sandboxd 碰 docker.sock; 砍掉多租户与记忆是为了把深度放在 runtime 上。	第 19 章

每个回答都以“我怎么证明它”结尾: 说出对应的测试名和破坏补丁编号。

19 决策记录

编号	决策	理由	放弃的方案
ADR-01	从空仓库重来，不迁移旧 P0	P0 的主体是多租户控制平面，与 runtime 主线无关且给每张新表加税。	在 P0 上删减
ADR-02	PostgreSQL SKIP LOCKED 做分发	单一事实来源，领取与状态变更同事务，无 outbox/双写。	Redis Streams + outbox + sweeper
ADR-03	attempt 作为 fencing token，租约过期只由 reaper 裁决	写入只比较整数，不比较时钟；避免多进程时钟不一致。	写入时比较 <code>lease_expires_at > now()</code>
ADR-04	checkpoint fencing 用数据库触发器	与写入同事务、不依赖 saver 内部实现、内核无法禁用。	继承 <code>AsyncPostgresSaver</code> ；让 checkpoint 经 Go 转写
ADR-05	<code>thread_id = run_id</code> ，Session 拥有工作区	一次 Run 一条执行线，续跑语义简单；多轮对话以多个 Run 共享工作区表达。	<code>thread_id = session_id</code>
ADR-06	Go/Python 进程在宿主机运行，compose 只放基础设施	工作区路径在 api 与容器挂载间一致； <code>kill -9 / SIGSTOP</code> 演示直接。	全部进 compose（需处理 docker.sock 与路径映射）
ADR-07	不宣称 exactly-once，非幂等工具报告 interrupted	副作用在数据库之外，诚实的语义比错误的承诺更可靠。	自动重跑全部工具
ADR-08	工具执行生命周期绑定请求 ctx	发起者消失即停止，避免孤儿进程与过期结果。	后台执行、结果异步回写
ADR-09	模型看到全部配置工具，网关做拒绝	拒绝路径真实发生，可测可演示；也符合“提示不是约束”。	只给模型有效工具
ADR-10	一个内核进程同时只跑一个 Run	心跳、取消、连接与 fence 上下文都是一对一，最易理解；并行靠多进程。	单进程多 Run 协程
ADR-11	不做多租户、记忆、Goal、审批、控制台	把有限精力放在可证明的 runtime 深度上。	—

附录 A 错误码

code	HTTP	含义
unauthorized	401	缺少或错误的 Bearer token
invalid_request	400	JSON 解析失败、未知字段、参数越界
invalid_config	400	AgentConfig 校验失败 (message 含字段路径)
not_found	404	资源不存在
agent_exists / agent_unpublished	409	重名 / Agent 无当前版本
session_busy	409	Session 已有活跃 Run
run_finished / invalid_transition	409	对终态 Run 取消 / 非法的完成状态
lease_lost	409	fence 条件不成立
tool_call_in_progress / tool_call_conflict	409	同一调用执行中 / 同 id 不同参数
internal	500	未分类错误

工具结果 error_code (返回给模型, 不使 Run 失败)

tool_not_in_agent、tool_not_allowed、invalid_args、path_outside_workspace、not_found、binary_file、skill_not_in_agent、skill_changed、sandbox_unavailable、executor_lost、cancelled。

Run error_code

no_result、max_attempts_exceeded、timeout、budget_exceeded、loop_detected、llm_unavailable、llm_bad_request、kernel_error。

数据库自定义 SQLSTATE

CD001：checkpoint 写入缺少 fence 上下文或 thread 不匹配；CD409：fence 条件不成立。

附录 B 配置项与默认值

变量	默认	说明 (引入步骤)
CD_PUBLIC_ADDR / CD_INTERNAL_ADDR	:7310 / 127.0.0.1:7312	A1 / B2
CD_DATABASE_URL	— (必填)	cd_app 连接串 (A2)
CD_MIGRATE_DATABASE_URL	—	cd_migrate, 仅 migrate 命令 (A2)
CD_API_KEY / CD_INTERNAL_TOKEN	— (必填)	A2 / B2
CD_REDIS_URL	redis://127.0.0.1:7341	B4
CD_WORKSPACES_DIR	./var/workspaces	B1
CD_LEASE_SECONDS / CD_HEARTBEAT_SECONDS	30 / 10	B2 / C1
CD_CLAIM_MAX_WAIT_MS	20000	B2
CD_REAPER_INTERVAL_MS	5000	C2
CD_TOOL_OUTPUT_MAX_BYTES	16384	D1
CD_SANDBOX_SOCKET	./var/run/sandboxd.sock	D3
CD_SANDBOX_IMAGE	cakerdesk-sandbox:dev	D3
CD_SANDBOX_UID / CD_SANDBOX_GID	sandboxd 进程的 uid/gid	D3
CD_SANDBOX_IDLE_SECONDS	600	D3
CD_SANDBOX_MEMORY_MB / CPUS / PIDS	512 / 1 / 256	D3
CD_SKILLS_DIR	./skills	D3 / E1
内核		
CD_KERNEL_API_URL	http://127.0.0.1:7312	B3
CD_KERNEL_DATABASE_URL	—	cd_kernel 连接串 (B3)
CD_KERNEL_SETUP_DATABASE_URL	—	cd_migrate, cd-kernel setup (B3)
CD_LLM_BASE_URL / CD_LLM_API_KEY	http://127.0.0.1:7330/v1 / mock	换成任意 OpenAI 兼容端点即可用真模型 (B3)
CD_EVENT_FLUSH_MS / CD_EVENT_BATCH	500 / 20	B4
CD_RETRY_BASE_MS / CD_RETRY_MAX	1000 / 5	E2

附录 C 术语表

术语	含义
Run	一次任务执行；绑定一个 AgentVersion 与一个 Session；有唯一的 LangGraph thread。
attempt	Run 被领取的次数，同时是 fencing token。
租约 lease	<code>(lease_owner, lease_expires_at)</code> ，持有者才能推进 Run。
fence / fencing	写入时校验 <code>(run_id, attempt, worker_id, status=running)</code> ，拒绝过时持有者。
僵尸 zombie	租约已被接管但自己不知道、仍在运行的旧内核。
reaper	回收过期租约、处理超时与取消收尾的进程。
checkpoint	LangGraph 在每个节点结束时保存的完整图状态。
重放 replay	工具调用已有完成记录时，直接返回存储的结果而不执行。
interrupted	工具调用开始执行但执行者消失，副作用未知。
executor_id	api 进程实例的标识，用于判断执行者是否存活。
有效工具集合	<code>config.tools ∩ (BASELINE ∪ 已加载 Skill 的 allowed-tools)</code> 。
破坏测试 sabotage	故意移除一条保证，验证对应测试会失败。

附录 D 按步骤的学习资料

步骤	资料
A1	Go 官方 Routing Enhancements for Go 1.22 ; uv 文档 ; Docker Compose healthcheck 文档。
A2	PostgreSQL 文档: Privileges 、 ALTER DEFAULT PRIVILEGES ; pgx v5 README ; goose README 。
A3	OpenAI Function calling 指南 (消息格式部分) 。
B1	PostgreSQL: Partial Indexes 、 错误码附录 。
B2	PostgreSQL The Locking Clause (SKIP LOCKED) ; 搜索 “Postgres SKIP LOCKED queue” 的实践文章。
B3	LangGraph 文档: Graph API 、 Persistence ; langgraph-checkpoint-postgres README 。
B4	MDN Using server-sent events ; Redis Pub/Sub 文档 (投递语义部分) 。
C1–C3	Martin Kleppmann How to do distributed locking (fencing token) ; 《数据密集型应用系统设计》第 8 章; PostgreSQL Trigger Functions 、 Explicit Locking 。
D1	Go Traversal-resistant file APIs (os.Root) 。
D2	Stripe Designing robust and predictable APIs with idempotency 。
D3–D4	Docker Engine security 、 <code>docker run</code> 资源限制参考 ; <code>man setsid</code> 、 <code>man 2 kill</code> (负 PID) 。
E1	Anthropic Agent Skills 文档 (SKILL.md 格式) 。
E2–E3	AWS Architecture Blog “ Exponential Backoff And Jitter ” ; LangGraph 文档中关于 <code>RemoveMessage</code> 与消息管理的部分。